Projet

BiG MART sales Prediction

Machine Learning

Data Mining

Réalisé Par : ABTANE Ayoub

PLAN

- Contexte du projet
- Data Cleaning et Preprocessing
- DataMinig
- Machine learning :
  - Réduction de dimmentionalité
  - Validation Croisée
  - Evaluation des modèles
- Streamlit





Contexte du Projet

Big Mart Sales Prediction

Data Preprocessing

Big Mart Sales Prediction

Aperçu sur les Données

Item_Identifier	**Unique product ID**
Item_Weight	Weight of product
Item_Fat_Content	Whether the product is low fat or not
Item_Visibility	The % of total display area of all products in a store allocated to the particular product
Item_Type	The category to which the product belongs
Item_MRP	Maximum Retail Price (list price) of the product
Outlet_Identifier	Unique store ID
Outlet_Establishment_Year	The year in which store was established
Outlet_Size	The size of the store in terms of ground area covered
Outlet_Location_Type	The type of city in which the store is located
Outlet_Type	Whether the outlet is just a grocery store or some sort of supermarket
Item_Outlet_Sales	Sales of the product in the particulat store. This is the outcome variable to be predicted.

Train et Test Data

```
df_train.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type	Outlet_Type
0	FDA15	9.30	Low Fat	0.02	Dairy	249.81	OUT049	1999	Medium	Tier 1	Supermarket Type1
1	DRC01	5.92	Regular	0.02	Soft Drinks	48.27	OUT018	2009	Medium	Tier 3	Supermarket Type2
2	FDN15	17.50	Low Fat	0.02	Meat	141.62	OUT049	1999	Medium	Tier 1	Supermarket Type1
3	FDX07	19.20	Regular	0.00	Fruits and Vegetables	182.09	OUT010	1998	NaN	Tier 3	Grocery Store
4	NCD19	8.93	Low Fat	0.00	Household	53.86	OUT013	1987	High	Tier 3	Supermarket Type1

Shape des données :

```
df_train.shape
```

```
(8523, 12)
```

```
df_test.shape
```

```
(5681, 11)
```

Concaténation du Train et Test Data

```
[ ] df_train["source"]="train"
    df_test["source"]="test"
    df=pd.concat([df_train,df_test],ignore_index=True)
    df.head()
```

Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type	Outlet_Type	Item_Outlet_Sales	source
9.30	Low Fat	0.02	Dairy	249.81	OUT049	1999	Medium	Tier 1	Supermarket Type1	3735.14	train
5.92	Regular	0.02	Soft Drinks	48.27	OUT018	2009	Medium	Tier 3	Supermarket Type2	443.42	train
17.50	Low Fat	0.02	Meat	141.62	OUT049	1999	Medium	Tier 1	Supermarket Type1	2097.27	train
19.20	Regular	0.00	Fruits and Vegetables	182.09	OUT010	1998	NaN	Tier 3	Grocery Store	732.38	train
8.93	Low Fat	0.00	Household	53.86	OUT013	1987	High	Tier 3	Supermarket Type1	994.71	train

Shape du df :

```
[ ] df.shape

    (14204, 13)
```

Aperçu sur df

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14204 entries, 0 to 14203
Data columns (total 13 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   Item_Identifier            14204 non-null  object 
 1   Item_Weight                11765 non-null  float64
 2   Item_Fat_Content           14204 non-null  object 
 3   Item_Visibility            14204 non-null  float64
 4   Item_Type                  14204 non-null  object 
 5   Item_MRP                   14204 non-null  float64
 6   Outlet_Identifier          14204 non-null  object 
 7   Outlet_Establishment_Year  14204 non-null  int64  
 8   Outlet_Size                10188 non-null  object 
 9   Outlet_Location_Type       14204 non-null  object 
 10  Outlet_Type                14204 non-null  object 
 11  Item_Outlet_Sales          8523 non-null   float64
 12  source                     14204 non-null  object 
dtypes: float64(4), int64(1), object(8)
memory usage: 1.4+ MB
```

Les données manquantes

```
df.isnull().sum()
```

```
Item_Identifier                 0
Item_Weight                  2439
Item_Fat_Content                0
Item_Visibility                 0
Item_Type                       0
Item_MRP                        0
Outlet_Identifier               0
Outlet_Establishment_Year       0
Outlet_Size                  4016
Outlet_Location_Type            0
Outlet_Type                     0
Item_Outlet_Sales            5681
source                          0
dtype: int64
```

Après traitement de l'Item_Weight :

Item_Weight

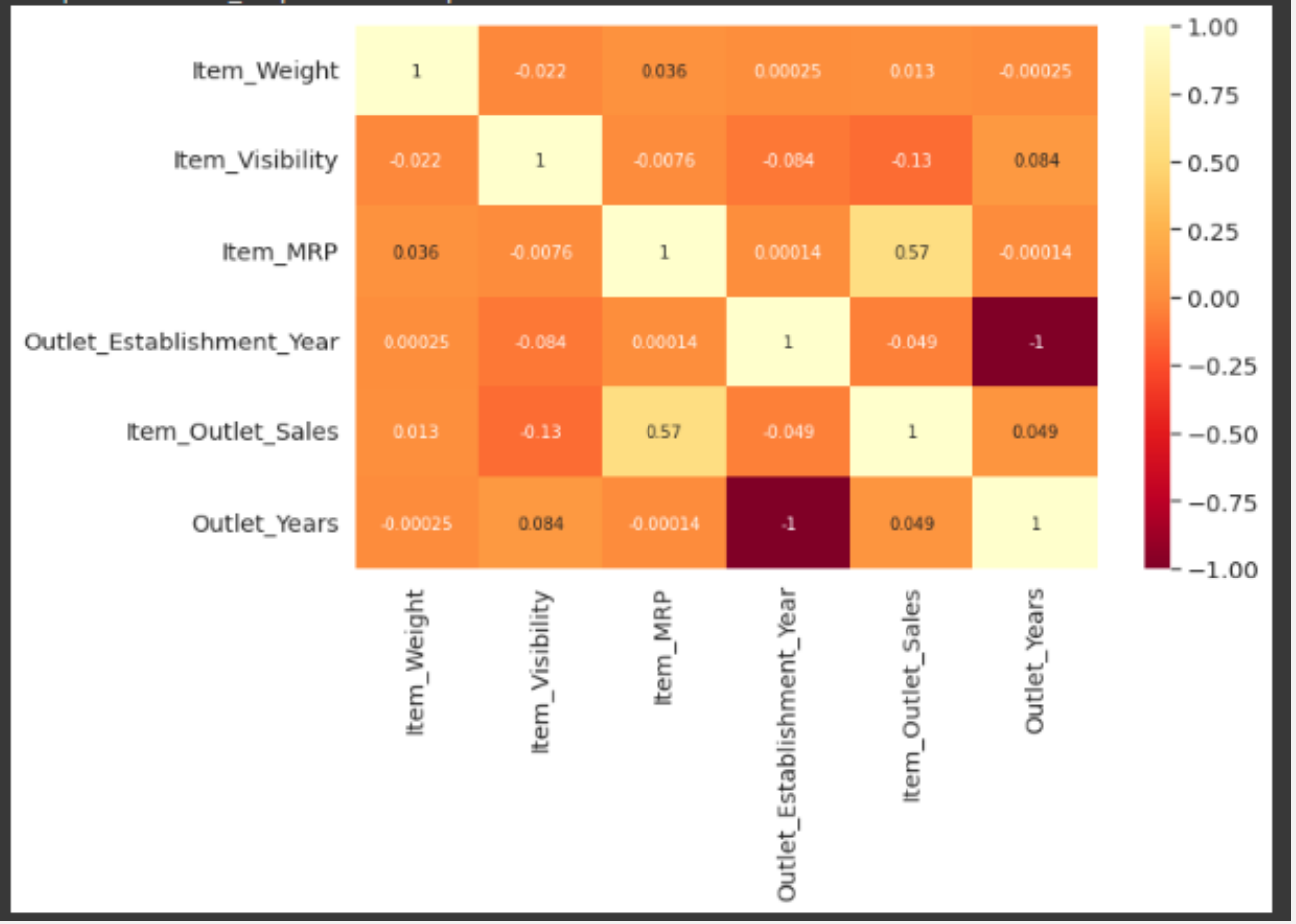
```
[20]
    #lets impute the item weight by mean of item weight groupped by item identifier
    item_weight_means=df.groupby(["Item_Identifier"]).agg({"Item_Weight":"mean"})
    item_weight_means
    for index in item_weight_means.index:
        null_condition=df.loc[df["Item_Identifier"]==index,["Item_Weight"]].isnull()
        for index_2 in null_condition.index:
            if null_condition.loc[index_2][0]==True:
                df.loc[index_2,["Item_Weight"]]=item_weight_means.loc[index][0]
    df.isnull().sum()
```

```
Item_Identifier                 0
Item_Weight                     0
Item_Fat_Content                0
Item_Visibility                 0
Item_Type                       0
Item_MRP                        0
Outlet_Identifier               0
Outlet_Establishment_Year       0
Outlet_Size                  4016
Outlet_Location_Type            0
Outlet_Type                     0
Item_Outlet_Sales            5681
source                          0
dtype: int64
```

Matrice de Corrélation :

```
plt.figure(figsize=(10,6))
sns.heatmap(dfcor,cmap='YlOrRd_r',annot=True)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f43a2585bd0>
```




```
corr_matrix = df.corr()
corr_matrix["Item_Outlet_Sales"].sort_values(ascending=True)
```

```
Item_Visibility             -0.13
Outlet_Establishment_Year   -0.05
Item_Weight                  0.01
Outlet_Years                 0.05
Item_MRP                     0.57
Item_Outlet_Sales            1.00
Name: Item_Outlet_Sales, dtype: float64
```

One Hot Encoding

```
df_bigmart=pd.get_dummies(df_new,columns=["Item_Fat_Content","Item_Type","Outlet_Location_Type","Outlet_Size","Outlet_Type"])
df_bigmart.head()
```

	Item_Identifier	Item_Weight	Item_Visibility	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Item_Outlet_Sales	source	Outlet_Years	Item_Fat_Content_Low Fat	Item_Fat
0	FDA15	9.30	0.02	249.81	OUT049	1999	3735.14	train	10	1	
1	DRC01	5.92	0.02	48.27	OUT018	2009	443.42	train	0	0	
2	FDN15	17.50	0.02	141.62	OUT049	1999	2097.27	train	10	1	
3	FDX07	19.20	0.02	182.09	OUT010	1998	732.38	train	11	0	
4	NCD19	8.93	0.01	53.86	OUT013	1987	994.71	train	22	1	

```
#On retire ["Item_Identifier","Outlet_Identifier","source"]
df_bigmart.drop(columns=["Item_Identifier","Outlet_Identifier"],inplace=True)
#on seprate le train et le  test
df_train=df_bigmart.loc[df_bigmart["source"]=="train"]
df_test=df_bigmart.loc[df_bigmart["source"]=="test"]
df_test.reset_index(drop=True,inplace=True)
print(df_train.head(2))
print(df_test.head(2))
```

```
   Item_Type_Health and Hygiene  Item_Type_Household  Item_Type_Meat  \
0                             0                    0               0
1                             0                    0               0

   Item_Type_Others  Item_Type_Seafood  Item_Type_Snack Foods  \
0                 0                  0                      0
1                 0                  0                      0

   Item_Type_Soft Drinks  Item_Type_Starchy Foods  \
0                      0                        0
1                      1                        0

   Outlet_Location_Type_Tier 1  Outlet_Location_Type_Tier 2  \
0                            1                            0
```

Standard Scaler

```
#On retire Item_Outlet_Sales du test dataset puisque c'est elle qui devra etre prédite
df_test.drop(columns=["Item_Outlet_Sales"],inplace=True)

#on retire source column du df_train et df_test
df_train.drop(columns=["source"],inplace=True)
df_test.drop(columns=["source"],inplace=True)

#on sépare input and output from df_train
df_x=df_train.drop(columns=["Item_Outlet_Sales"])
y=df_train[["Item_Outlet_Sales"]]
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x = sc.fit_transform(df_x)
x=pd.DataFrame(x,columns=df_x.columns)
x.head()
```

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Years	Item_Fat_Content_Low Fat	Item_Fat_Content_Regular	Item_Type_Baking Goods	Item_Type_Breads	Item_Type_Breakfast	Item_Type_Canne
0	-0.77	-1.33	1.75	-0.14	0.74	-0.74	-0.29	-0.17	-0.11	-0.2
1	-1.50	-1.19	-1.49	-1.33	-1.35	1.35	-0.29	-0.17	-0.11	-0.2
2	1.00	-1.30	0.01	-0.14	0.74	-0.74	-0.29	-0.17	-0.11	-0.2
3	1.36	-1.25	0.66	-0.02	-1.35	1.35	-0.29	-0.17	-0.11	-0.2
4	-0.85	-1.63	-1.40	1.29	0.74	-0.74	-0.29	-0.17	-0.11	-0.2

```
[ ] x.shape
```

```
(8523, 33)
```

Séparation Train/Test

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=.20,random_state=3)
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(6818, 33)
(1705, 33)
(6818, 1)
(1705, 1)
```

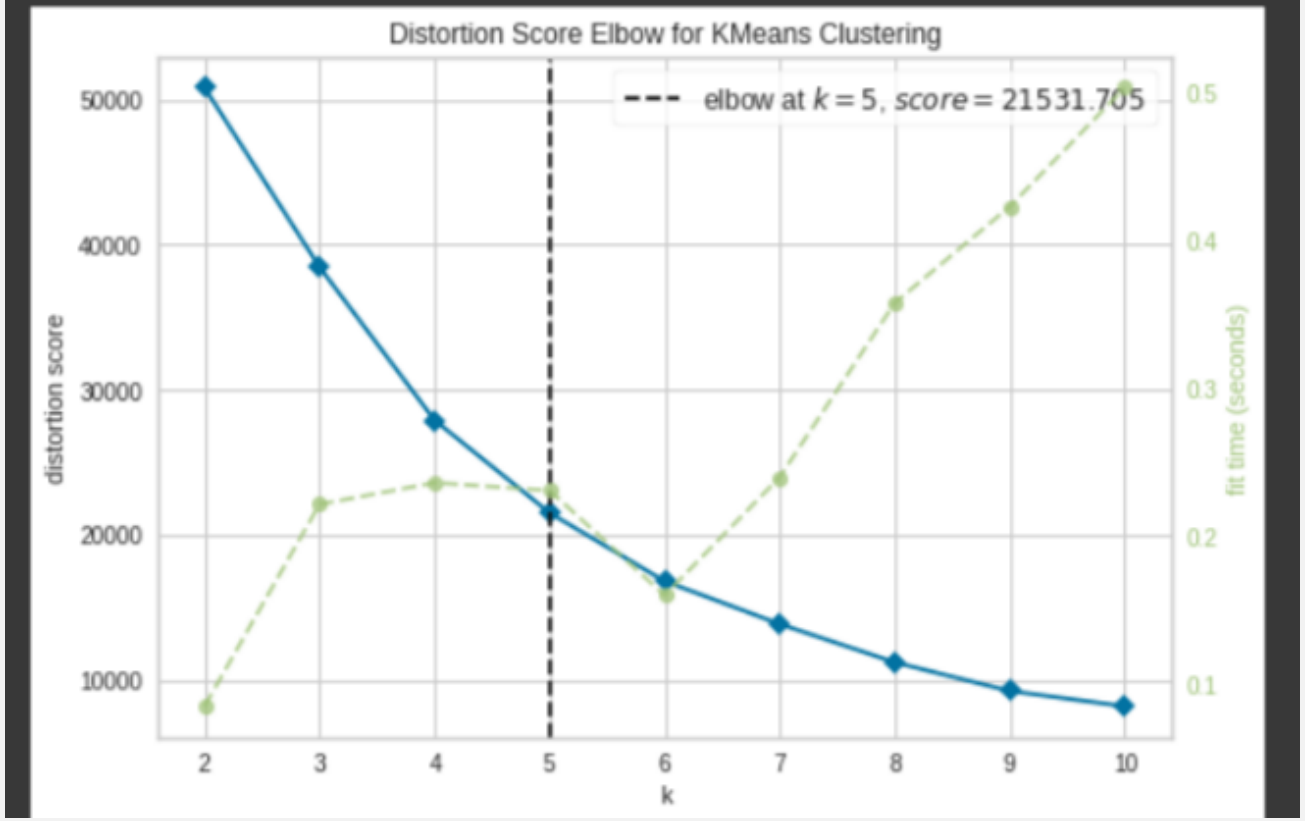
DataMining

Big Mart Sales Prediction

K-means

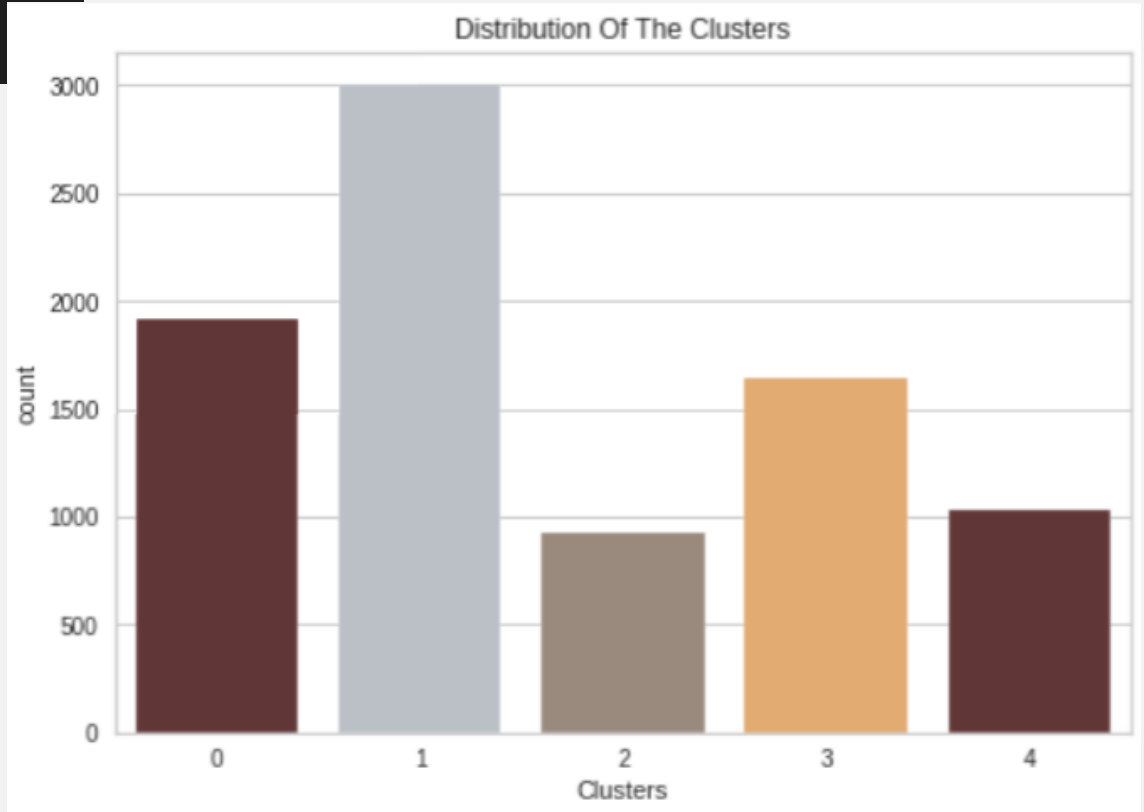
```
# Quick examination of elbow method to find numbers of clusters to make.
print('Elbow Method to determine the number of clusters to be formed:')
Elbow_M = KElbowVisualizer(KMeans(), k=10)
Elbow_M.fit(PCA_ds)
Elbow_M.show()
```

```
Elbow Method to determine the number of clusters to be formed:
```



K-means

```
#Initiating the Agglomerative Clustering model
AC = AgglomerativeClustering(n_clusters=5)
# fit model and predict clusters
yhat_AC = AC.fit_predict(PCA_ds)
PCA_ds["Clusters"] = yhat_AC
#Adding the Clusters feature to the orignal dataframe.
df_train["Clusters"]= yhat_AC
```



K-means

```
# k-means with some arbitrary k
kmeans = KMeans(n_clusters=5, max_iter=100)
kmeans.fit(PCA_ds)
```

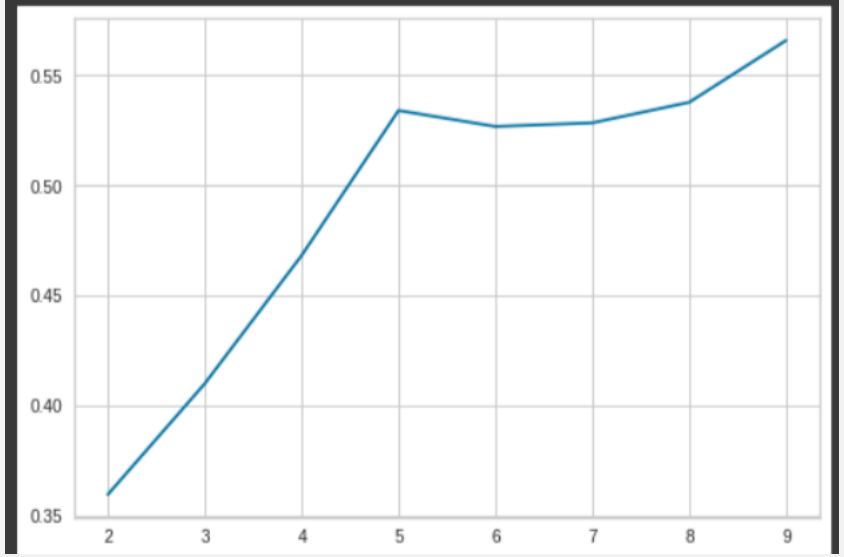
```
KMeans(max_iter=100, n_clusters=5)
```

```
kmeans.labels_
```

```
array([0, 4, 0, ..., 0, 4, 0], dtype=int32)
```

```
ss = []
for k in range(2,10):
    kmeans = KMeans(n_clusters = k).fit(PCA_ds)
    ss.append([k, silhouette_score(PCA_ds, kmeans.labels_)])

plt.plot(pd.DataFrame(ss)[0], pd.DataFrame(ss)[1]);
```




```
# silhouette analysis
range_n_clusters = [2, 3, 4, 5, 6, 7, 8]

for num_clusters in range_n_clusters:

    # intialise kmeans
    kmeans = KMeans(n_clusters=num_clusters, max_iter=50)
    kmeans.fit(PCA_ds)

    cluster_labels = kmeans.labels_

    # silhouette score
    silhouette_avg = silhouette_score(PCA_ds, cluster_labels)
    print("For n_clusters={0}, the silhouette score is {1}".format(num_clusters, silhouette_avg))
```

```
For n_clusters=2, the silhouette score is 0.3595853134808709
For n_clusters=3, the silhouette score is 0.40995350359345656
For n_clusters=4, the silhouette score is 0.46819619658911105
For n_clusters=5, the silhouette score is 0.5340455688300643
For n_clusters=6, the silhouette score is 0.5350646366445054
For n_clusters=7, the silhouette score is 0.5284377635866497
For n_clusters=8, the silhouette score is 0.5349244955955658
```

K-means

```
[ ]  ## Let's make a copy of our original data before using it for analysis
     orginal_data = df_train.copy()
```

```
[ ]  df_train.head()
```

	Item_Weight	Item_Visibility	Item_MRP	Item_Outlet_Sales	Outlet_Years	Item_Fat_Content_Low Fat	Item_Fat_Content_Regular	Item_Type_Baking Goods	Item_Type_Breads	Item_Typ
0	9.30	0.13	249.81	3735.14	10	1	0	0	0	
1	5.92	0.14	48.27	443.42	0	0	1	0	0	
2	17.50	0.13	141.62	2097.27	10	1	0	0	0	
3	19.20	0.13	182.09	732.38	11	0	1	0	0	
4	8.93	0.10	53.86	994.71	22	1	0	0	0	



K-means

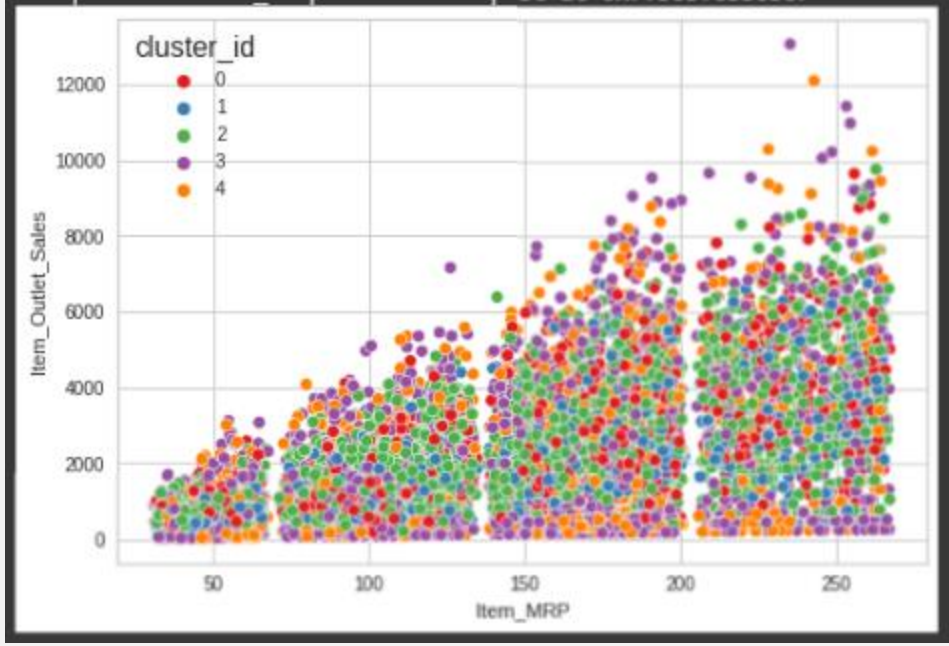
```
# Let's add the cluster Ids to the original data
df_km = pd.concat([df_train.reset_index().drop('index', axis = 1), pd.Series(kmean.labels_)], axis = 1)
df_km.columns = ['Item_Weight', 'Item_Visibility', 'Item_MRP', 'Item_Outlet_Sales',
       'Outlet_Years', 'Item_Fat_Content_Low Fat', 'Item_Fat_Content_Regular',
       'Item_Type_Baking Goods', 'Item_Type_Breads', 'Item_Type_Breakfast',
       'Item_Type_Canned', 'Item_Type_Dairy', 'Item_Type_Frozen Foods',
       'Item_Type_Fruits and Vegetables', 'Item_Type_Hard Drinks',
       'Item_Type_Health and Hygiene', 'Item_Type_Household', 'Item_Type_Meat',
       'Item_Type_Others', 'Item_Type_Seafood', 'Item_Type_Snack Foods',
       'Item_Type_Soft Drinks', 'Item_Type_Starchy Foods',
       'Outlet_Location_Type_Tier 1', 'Outlet_Location_Type_Tier 2',
       'Outlet_Location_Type_Tier 3', 'Outlet_Size_High', 'Outlet_Size_Medium',
       'Outlet_Size_Small', 'Outlet_Type_Grocery Store',
       'Outlet_Type_Supermarket Type1', 'Outlet_Type_Supermarket Type2',
       'Outlet_Type_Supermarket Type3', 'Outlet__Years_of_operation',
       'Clusters','cluster_id']
df_km.head()
```

	Item_Weight	Item_Visibility	Item_MRP	Item_Outlet_Sales	Outlet_Years	Item_Fat_Content_Low Fat	Item_Fat_Content_Regular	Item_Type_Baking Goods	Item_Type_Breads	Item_Typ
0	9.30	0.13	249.81	3735.14	10	1	0	0	0	
1	5.92	0.14	48.27	443.42	0	0	1	0	0	
2	17.50	0.13	141.62	2097.27	10	1	0	0	0	
3	19.20	0.13	182.09	732.38	11	0	1	0	0	
4	8.93	0.10	53.86	994.71	22	1	0	0	0	

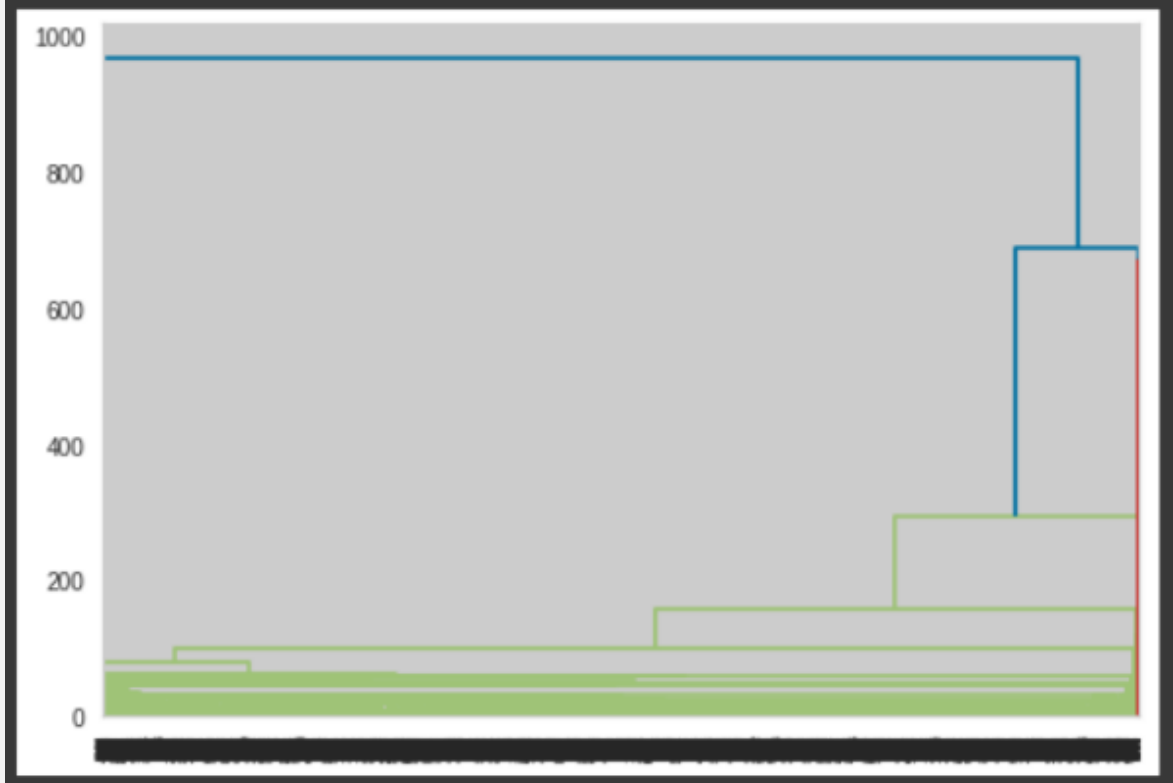
K-means

```
# Plot the Cluster with respect to the clusters obtained
sns.scatterplot(x = 'Item_MRP', y = 'Item_Outlet_Sales', hue ='cluster_id', legend = 'full', data = df_km, palette='Set1')
```

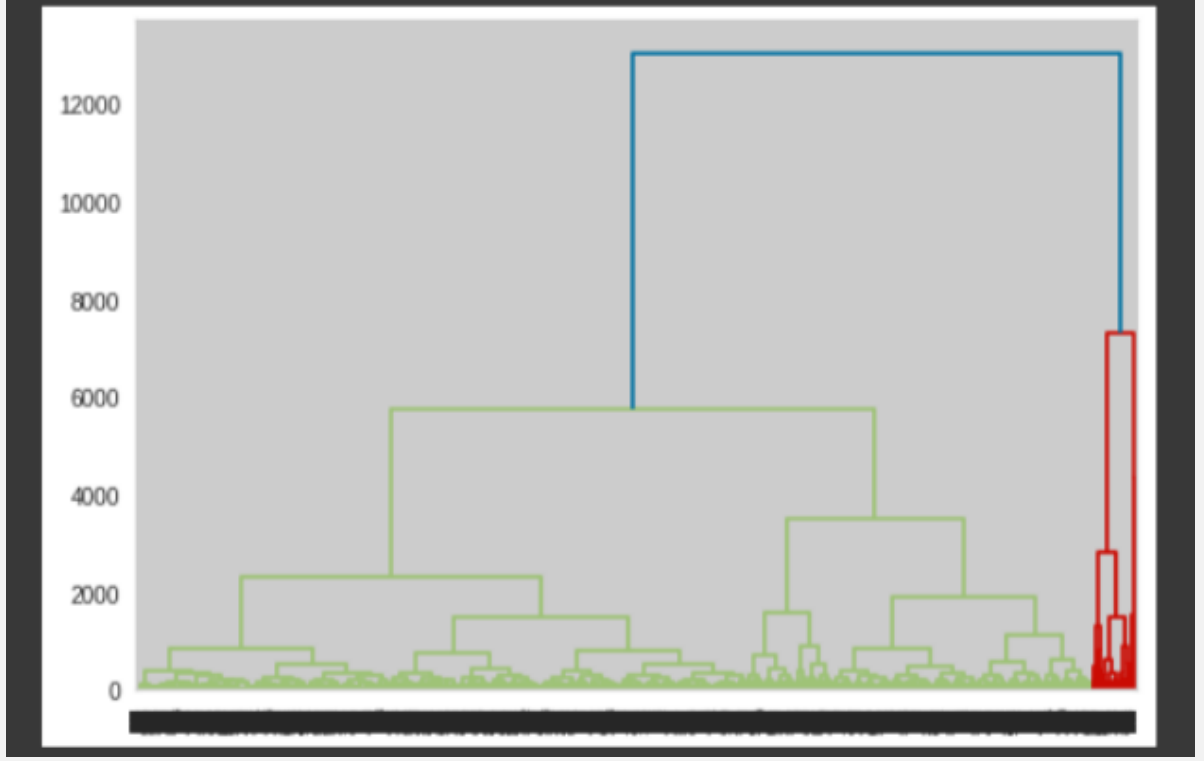
```
<matplotlib.axes._subplots.AxesSubplot at 0x7f303f6bb650>
```



```
# single linkage
mergings = linkage(df_km, method="single", metric='euclidean')
dendrogram(mergings)
plt.show()
```




```
# complete linkage
mergings = linkage(df_km, method="complete", metric='euclidean')
dendrogram(mergings)
plt.show()
```



DBSCAN

```
[93] from sklearn.cluster import DBSCAN
     from sklearn import metrics
     from sklearn.preprocessing import StandardScaler
```

```
[94] df_num = StandardScaler().fit_transform(df_x)
```

```
[95] from sklearn.metrics import silhouette_samples, silhouette_score
     range_n_clusters = [4, 5, 6, 7, 8, 9,10, 11, 12, 13]

     for num_clusters in range_n_clusters:

         db = DBSCAN(eps=0.8, min_samples= num_clusters, metric='euclidean')
         db.fit_predict(df_num)
         cluster_labels =db.labels_


         # silhouette score
         silhouette_avg = silhouette_score(df_num, cluster_labels)
         print("For n_clusters={0}, the silhouette score is {1}".format(num_clusters, silhouette_avg))
```

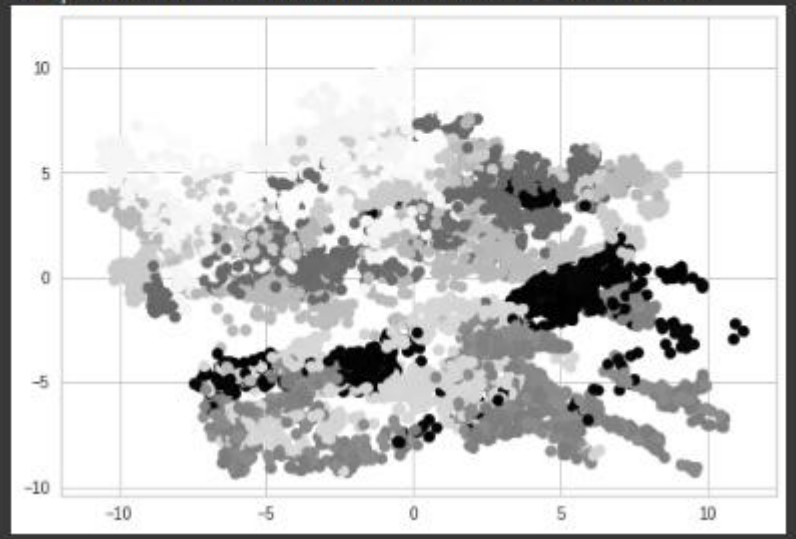
```
For n_clusters=4, the silhouette score is -0.1799697742488755
For n_clusters=5, the silhouette score is -0.2584310392826197
For n_clusters=6, the silhouette score is -0.30431472085070854
For n_clusters=7, the silhouette score is -0.3489574979721133
For n_clusters=8, the silhouette score is -0.36906636824571637
For n_clusters=9, the silhouette score is -0.362741265214683
For n_clusters=10, the silhouette score is -0.3527109468777174
For n_clusters=11, the silhouette score is -0.3354115043046214
For n_clusters=12, the silhouette score is -0.28559539815705
For n_clusters=13, the silhouette score is -0.17459620419726524
```

ISOMAP

```
from sklearn.manifold import Isomap

iso = Isomap(n_neighbors=20)
iso.fit(xnew)
X_iso = iso.transform(xnew)
plt.scatter(X_iso[:, 0], X_iso[:, 1], c=y)
```

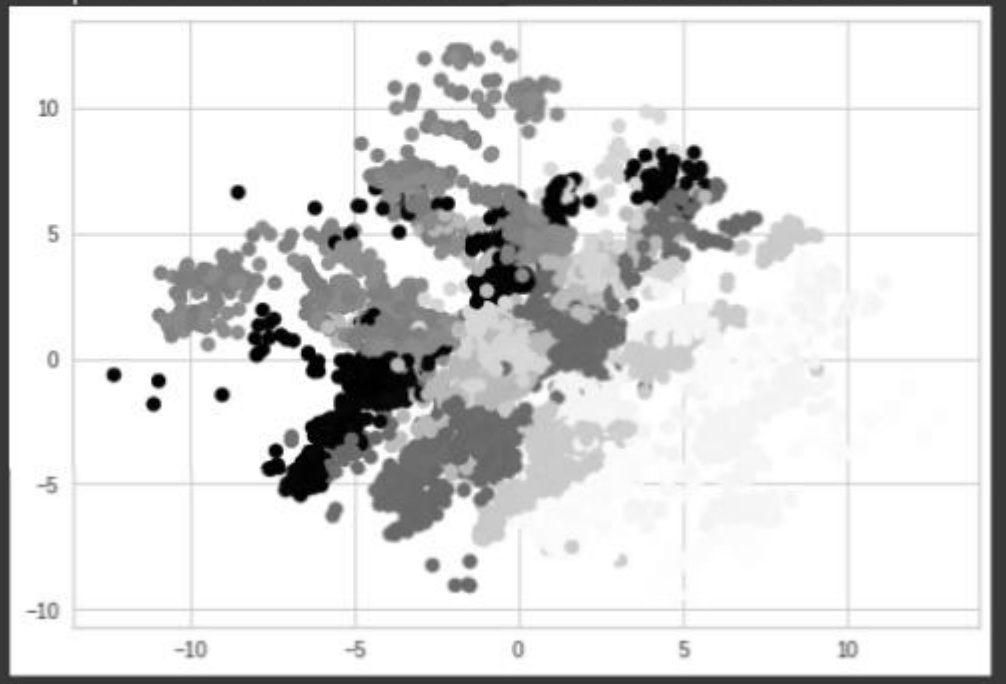
```
<matplotlib.collections.PathCollection at 0x7f2eac3d8390>
```




```
from sklearn.manifold import Isomap

iso = Isomap(n_neighbors=10)
iso.fit(xnew)
X_iso = iso.transform(xnew)
plt.scatter(X_iso[:, 0], X_iso[:, 1], c=y)
```

```
<matplotlib.collections.PathCollection at 0x7f2eabbf2d90>
```



TSNE

```
from sklearn.manifold import TSNE

tsne = TSNE()
tsne.fit(xnew)
```

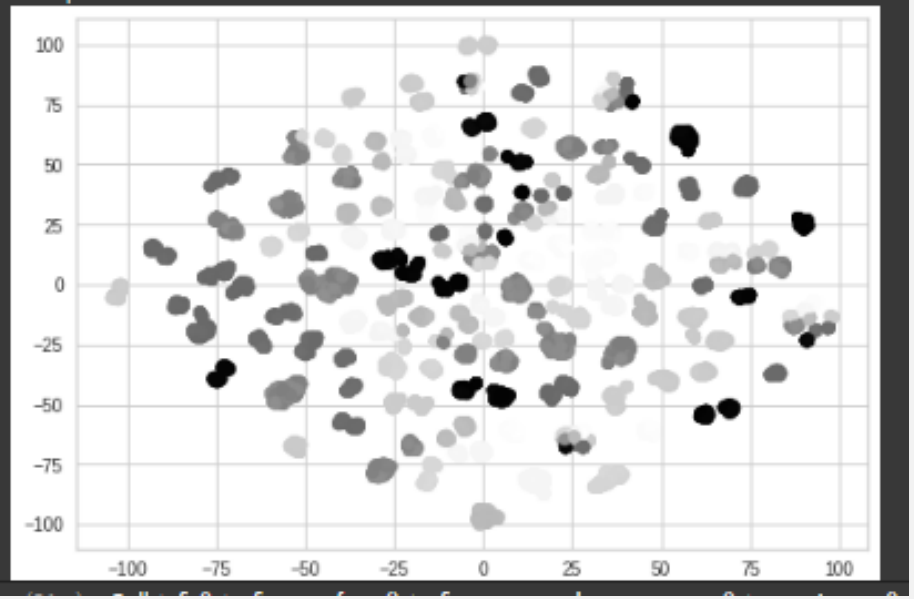
```
TSNE()
```

```
[109] from sklearn.manifold import TSNE
```

```
[113] X_tsne = TSNE().fit_transform(xnew)
```

```
[115]
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y)
```

```
<matplotlib.collections.PathCollection at 0x7f2eabf24c90>
```



Machine Learning

Big Mart Sales Prediction

Les modèles de régression utilisés

Méthodes d ’ensemble :

- Random forest
- Bagging

Méthodes séquentielles :

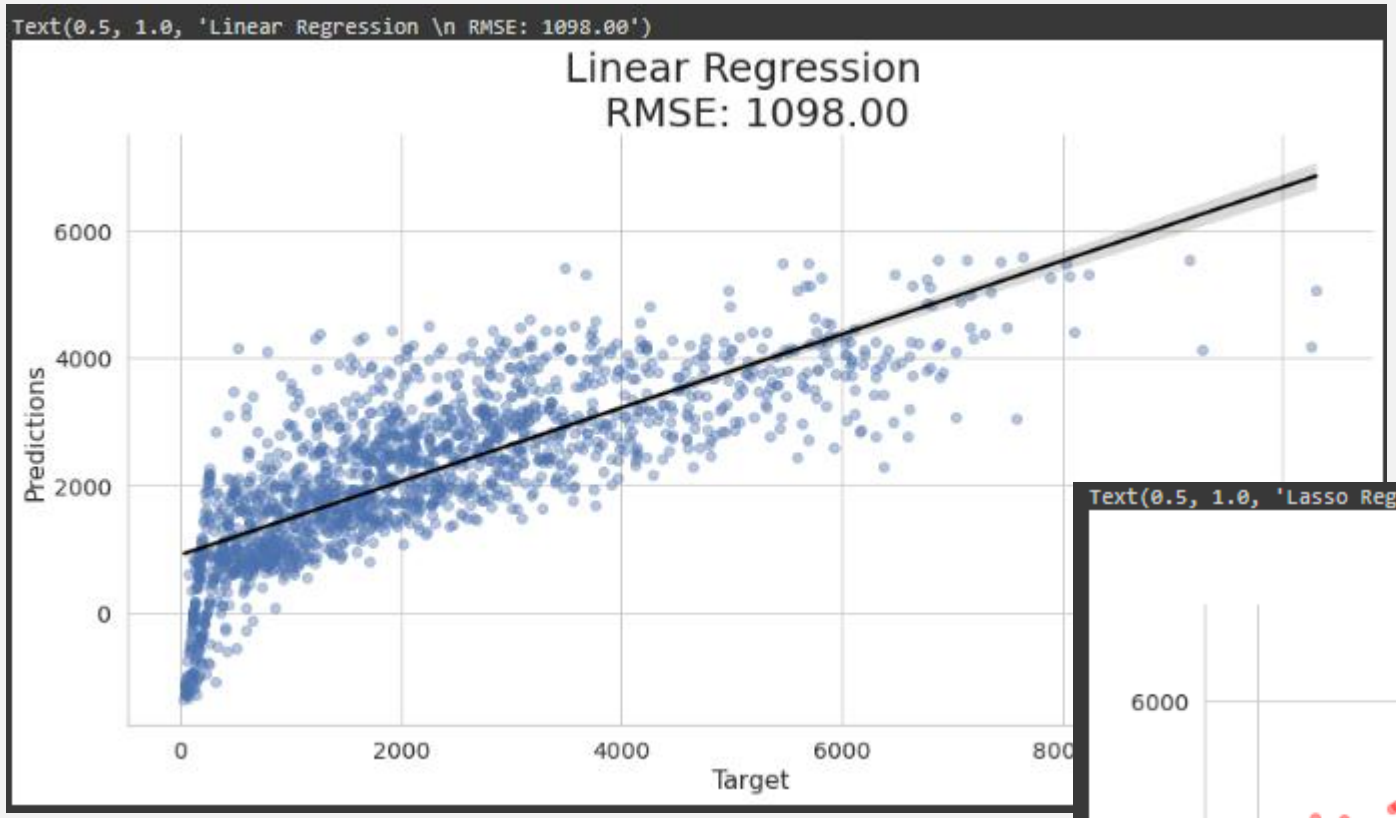
- Boosting
- Adaboost

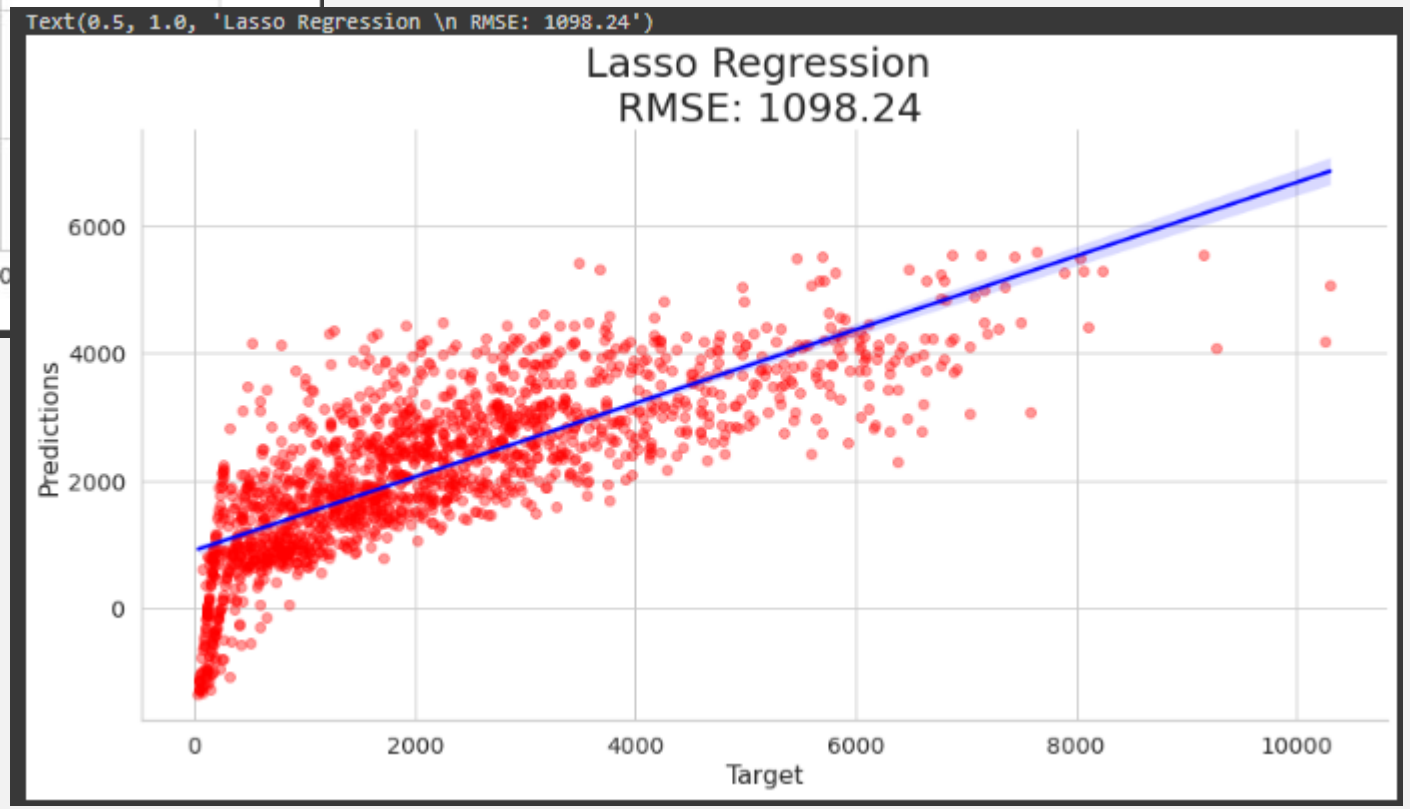
```
[ ]  from sklearn.linear_model import ElasticNet
     from sklearn.linear_model import SGDRegressor
     from sklearn.ensemble import BaggingRegressor
     from sklearn.ensemble import AdaBoostRegressor

     lr=LinearRegression()
     r=Ridge()
     l=Lasso(alpha=0.01)
     en=ElasticNet()
     sgd=SGDRegressor()
     svm=SVR()
     dtr=DecisionTreeRegressor()
     rf=RandomForestRegressor()
     br=BaggingRegressor()
     ada=AdaBoostRegressor()
     gbr=GradientBoostingRegressor()
     knn = KNeighborsRegressor()
```

Les performances des modèles avant ACP

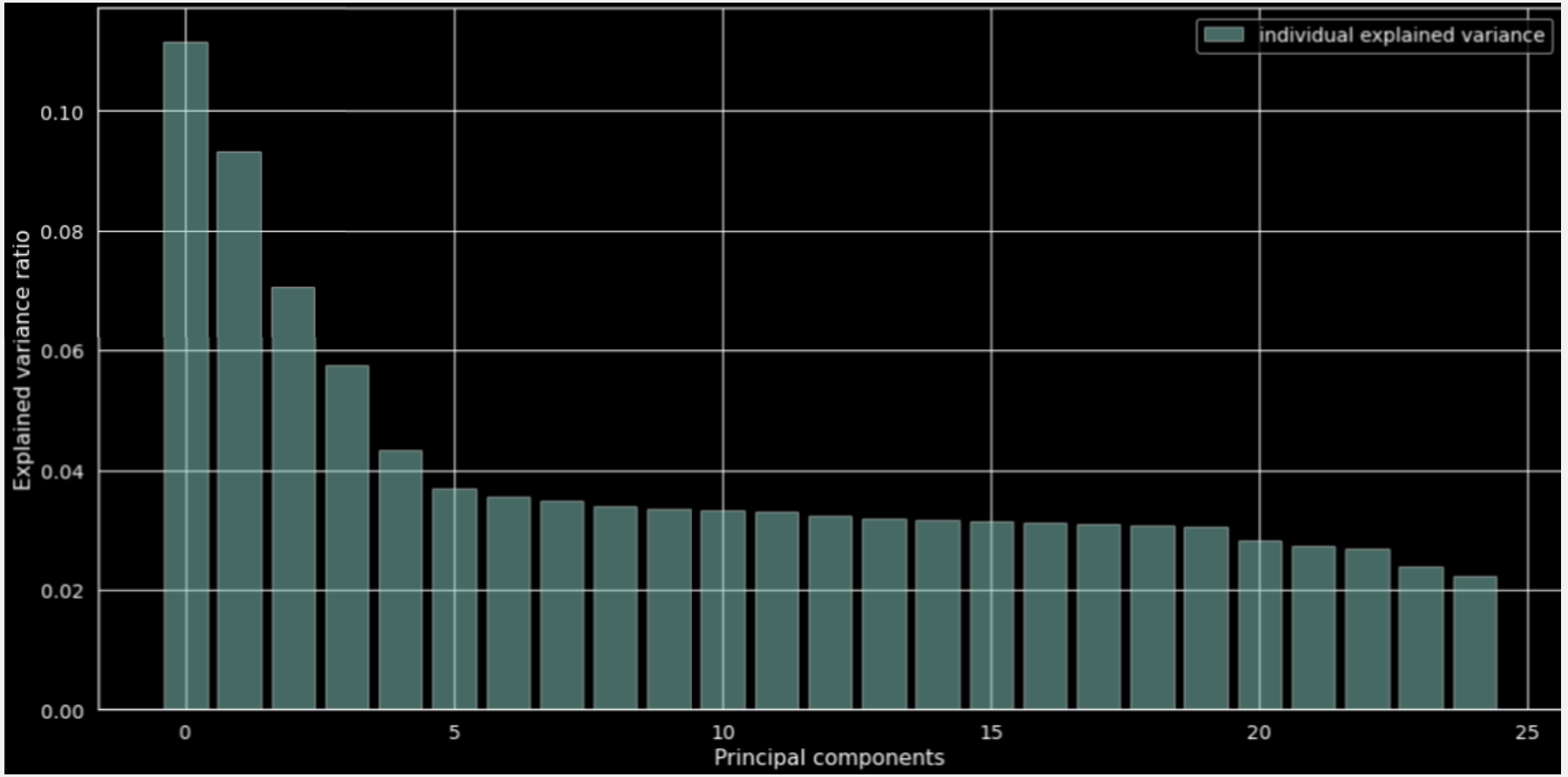
	Name	Error	R2	MAE	MSE	RMSE	CrossValidation	R2CVDiff
0	Lin Reg	0.55	0.59	817.58	1193241.69	1092.36	0.56	0.03
1	Lasso	0.55	0.59	817.53	1193112.67	1092.30	0.56	0.03
2	Ridge	0.55	0.59	817.52	1193160.49	1092.32	0.56	0.03
3	Gradient boost	0.63	0.62	737.41	1105890.36	1051.61	0.59	0.03
4	Decision Tree	1.00	0.22	1051.53	2250644.69	1500.21	0.16	0.07
5	Random Forest	0.94	0.58	781.34	1229604.21	1108.88	0.55	0.02
6	Kneigbor	0.64	0.48	880.39	1506967.21	1227.59	0.47	0.01
7	ElasticNet	0.50	0.52	874.32	1378132.59	1173.94	0.50	0.02
8	SGDRegressor	0.55	0.59	817.54	1192805.54	1092.16	0.56	0.03
9	SVR	-0.01	-0.00	1265.24	2908129.99	1705.32	-0.00	0.00
10	BaggingRegressor	0.91	0.53	810.13	1349804.89	1161.81	0.52	0.02
11	AdaBoostRegressor	0.47	0.49	991.25	1478835.36	1216.07	0.45	0.04
12	GradientBoostingRegressor	0.63	0.62	737.50	1106004.03	1051.67	0.59	0.03


Text(0.5, 1.0, 'Linear Regression \n RMSE: 1098.00')
Linear Regression
RMSE: 1098.00
Predictions
Target
0
2000
4000
6000
800

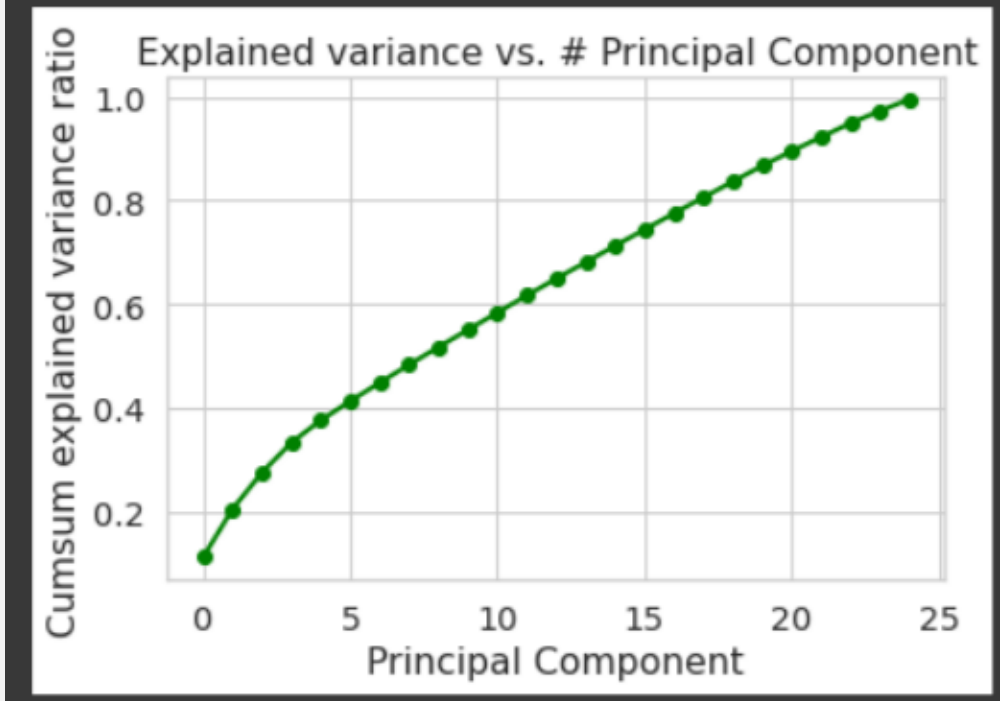

Text(0.5, 1.0, 'Lasso Regression \n RMSE: 1098.24')
Lasso Regression
RMSE: 1098.24
Predictions
Target
0
2000
4000
6000
8000
10000

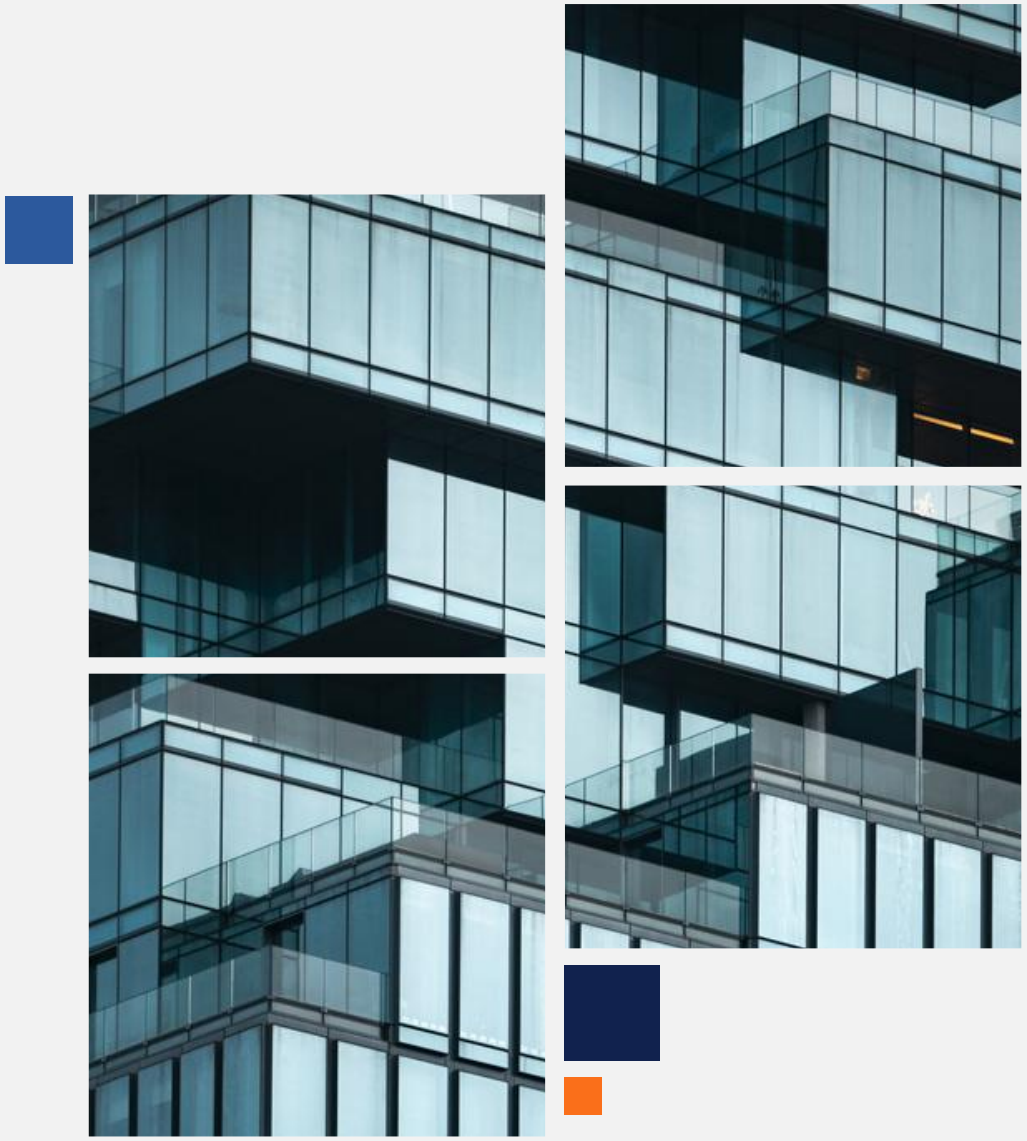
```
[ ]  #Lets use pca to reduce the dimensionality
     from sklearn.decomposition import PCA
     pca = PCA(n_components=25)
     x_pca=pca.fit_transform(x)
     print("variance :{}".format(np.sum(pca.explained_variance_ratio_)))

     variance :0.9957008361447693
```



```python
#cumul de variance expliquée
plt.plot(np.cumsum(pca.explained_variance_ratio_), 'o-', linewidth=2,
color='green')
plt.title("Explained variance vs. # Principal Component")
plt.ylabel("Cumsum explained variance ratio")
plt.xlabel("Principal Component")
plt.show()
```





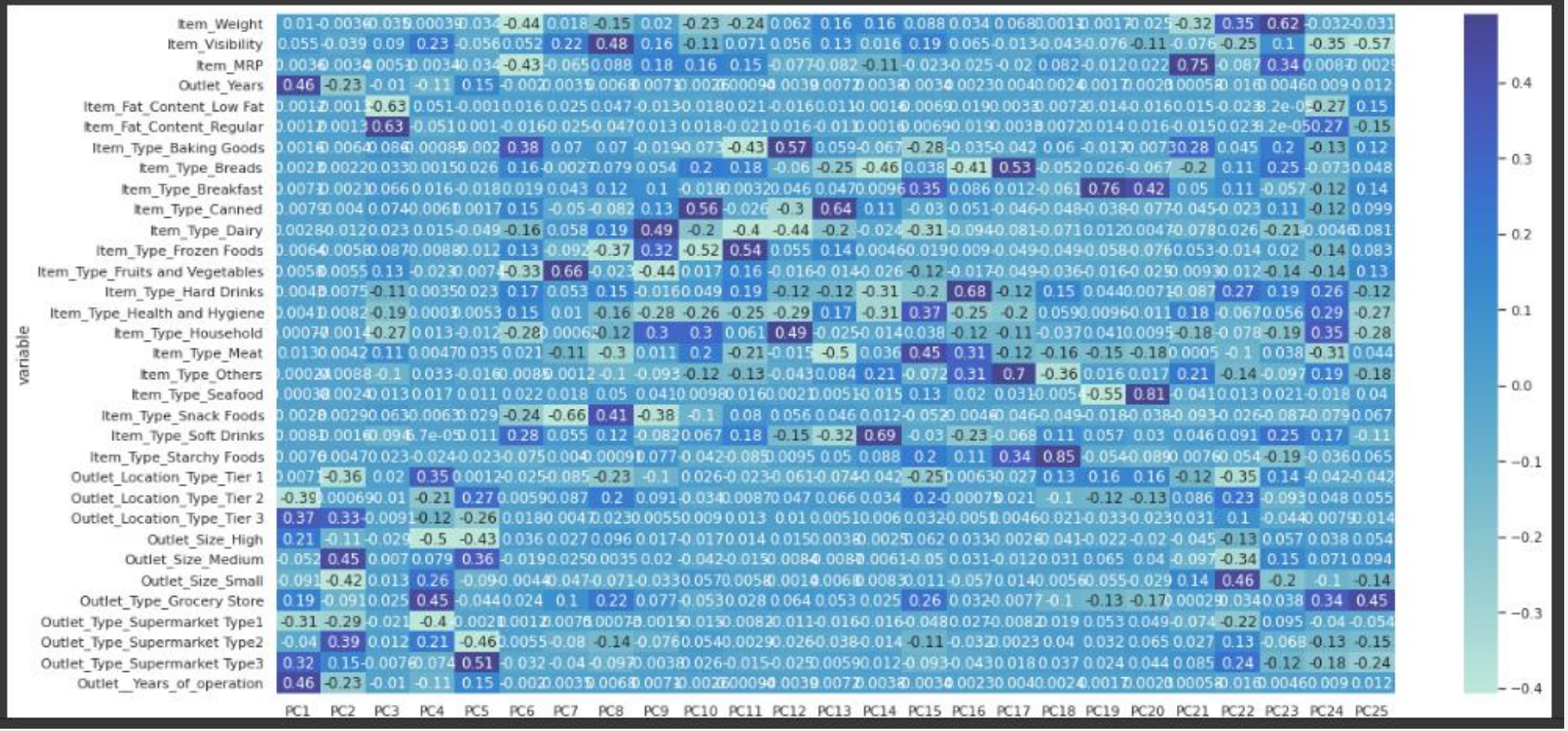
la matrice des corrélations entre les variables d'origine les composantes principales

```
loadings = pca.components_
num_pc = pca.n_features_
pc_list = ["PC"+str(i) for i in list(range(1, num_pc+1))]
loadings_df = pd.DataFrame.from_dict(dict(zip(pc_list, loadings)))
loadings_df['variable'] = df_x.columns.values
loadings_df = loadings_df.set_index('variable')
loadings_df
```

variable	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11	PC12	PC13	PC14	PC15	PC16	PC17	PC18	PC19	PC20	PC21	PC22	PC23	PC24	PC25
Item_Weight	0.01	-0.00	-0.04	0.00	-0.03	-0.44	0.02	-0.15	0.02	-0.23	-0.24	0.06	0.16	0.16	0.09	0.03	0.07	0.00	-0.00	-0.03	-0.32	0.35	0.62	-0.03	-0.03
Item_Visibility	0.06	-0.04	0.09	0.23	-0.06	0.05	0.22	0.48	0.16	-0.11	0.07	0.06	0.13	0.02	0.19	0.07	-0.01	-0.04	-0.08	-0.11	-0.08	-0.25	0.10	-0.35	-0.57
Item_MRP	-0.00	-0.00	0.01	-0.00	-0.03	-0.43	-0.07	0.09	0.18	0.16	0.15	-0.08	-0.08	-0.11	-0.02	-0.03	-0.02	0.08	-0.01	0.02	0.75	-0.09	0.34	0.01	-0.00
Outlet_Years	0.46	-0.23	-0.01	-0.11	0.15	-0.00	0.00	0.01	0.01	-0.00	0.00	-0.00	0.01	0.00	-0.00	0.00	0.00	0.00	0.00	0.00	0.00	-0.02	0.00	0.01	0.01
Item_Fat_Content_Low Fat	-0.00	-0.00	-0.63	0.05	-0.00	0.02	0.02	0.05	-0.01	-0.02	0.02	-0.02	0.01	-0.00	-0.01	0.02	0.00	-0.01	-0.01	-0.02	0.01	-0.02	-0.00	-0.27	0.15
Item_Fat_Content_Regular	0.00	0.00	0.63	-0.05	0.00	-0.02	-0.02	-0.05	0.01	0.02	-0.02	0.02	-0.01	0.00	0.01	-0.02	-0.00	0.01	0.01	0.02	-0.01	0.02	0.00	0.27	-0.15
Item_Type_Baking Goods	0.00	-0.01	0.09	-0.00	-0.00	0.38	0.07	0.07	-0.02	-0.07	-0.43	0.57	0.06	-0.07	-0.28	-0.03	-0.04	0.06	-0.02	-0.01	0.28	0.04	0.20	-0.13	0.12
Item_Type_Breads	0.00	0.00	0.03	0.00	0.03	0.16	-0.00	0.08	0.05	0.20	0.18	-0.06	-0.25	-0.46	0.04	-0.41	0.53	-0.05	0.03	-0.07	-0.20	0.11	0.25	-0.07	0.05
Item_Type_Breakfast	0.01	-0.00	0.07	0.02	-0.02	0.02	0.04	0.12	0.10	-0.02	0.00	0.05	0.05	0.01	0.35	0.09	0.01	-0.06	0.76	0.42	0.05	0.11	-0.06	-0.12	0.14
Item_Type_Canned	-0.01	0.00	0.07	-0.01	0.00	0.15	-0.05	-0.08	0.13	0.56	-0.03	-0.30	0.64	0.11	-0.03	0.05	-0.05	-0.05	-0.04	-0.08	-0.05	-0.02	0.11	-0.12	0.10
Item_Type_Dairy	0.00	-0.01	0.02	0.01	-0.05	-0.16	0.06	0.19	0.49	-0.20	-0.40	-0.44	-0.20	-0.02	-0.31	-0.09	-0.08	-0.07	0.01	0.00	-0.08	0.03	-0.21	-0.00	0.08
Item_Type_Frozen Foods	-0.01	-0.01	0.09	-0.01	-0.01	0.13	-0.09	-0.37	0.32	-0.52	0.54	0.05	0.14	0.00	-0.02	0.01	-0.05	-0.05	-0.06	-0.08	0.05	-0.01	0.02	-0.14	0.08
Item_Type_Fruits and Vegetables	0.01	0.01	0.13	-0.02	-0.01	-0.33	0.66	-0.02	-0.44	0.02	0.16	-0.02	-0.01	-0.03	-0.12	-0.02	-0.05	-0.04	-0.02	-0.02	-0.01	-0.01	-0.14	-0.14	0.13
Item_Type_Hard Drinks	-0.00	0.01	-0.11	0.00	0.02	0.17	0.05	0.15	-0.02	0.05	0.19	-0.12	-0.12	-0.31	-0.20	0.68	-0.12	0.15	0.04	0.01	-0.09	0.27	0.19	0.26	-0.12

Affichage graphique de la matrice des corrélations

```
[ ]  plt.figure(figsize=(20,10))
     sns.heatmap(loadings_df,annot=True,center=True,robust=True)
     plt.show()
```



Les performances des modèles après l ' ACP

	Name	Error	R2	MAE	MSE	RMSE	CrossValidation	R2CVDiff
0	Lin Reg	0.55	0.59	818.27	1199283.61	1095.12	0.56	0.03
1	Lasso	0.55	0.59	818.27	1199283.02	1095.12	0.56	0.03
2	Ridge	0.55	0.59	818.27	1199286.72	1095.12	0.56	0.03
3	Gradient boost	0.64	0.60	777.33	1148216.98	1071.55	0.59	0.01
4	Decision Tree	1.00	0.21	1069.41	2302846.92	1517.51	0.16	0.04
5	Random Forest	0.94	0.57	799.43	1240338.46	1113.70	0.55	0.02
6	Kneigbor	0.64	0.48	876.18	1495733.22	1223.00	0.47	0.01
7	ElasticNet	0.50	0.52	874.57	1379097.84	1174.35	0.50	0.02
8	SGDRegressor	0.55	0.58	821.32	1207021.60	1098.65	0.55	0.03
9	SVR	-0.01	-0.00	1265.09	2907935.28	1705.27	-0.00	0.00
10	BaggingRegressor	0.91	0.54	832.34	1344709.78	1159.62	0.51	0.02
11	AdaBoostRegressor	0.19	0.17	1342.61	2397455.91	1548.37	0.49	-0.32
12	GradientBoostingRegressor	0.64	0.60	777.42	1148168.46	1071.53	0.59	0.01

Réglage des hyperparamètres

Random Forest Regressor

```
[ ] from sklearn.model_selection import GridSearchCV
```

```
[ ] parameters ={
                 "n_estimato
                }
    GCV=GridSearchCV(RandomFo
    GCV.fit(x_train,y_train)
```

```
    GridSearchCV(cv=5, estima
                 param_grid={
```

```
[ ] GCV.best_params_
```

```
    {'n_estimators': 500}
```

```
[ ] mod=RandomForestRegressor
    mod.fit(x_train,y_train)
    pred=mod.predict(x_test)
    print(r2_score(y_test,pre
```

```
    0.5816858411425476
```

KNeighbors Regressor

```
[ ] parameters = {"n_neighbors
    GCV=GridSearchCV(KNeighbor
    GCV.fit(x_train,y_train)
```

```
    GridSearchCV(cv=5, estimato
                 param_grid={'
```

```
[ ] GCV.best_params_
```

```
    {'n_neighbors': 7}
```

```
[ ] mod2=KNeighborsRegressor(n_
    mod2.fit(x_train,y_train)
    pred=mod2.predict(x_test)
    print(r2_score(y_test,pred
```

```
    0.5067663828477093
```

Gradient Boosting Regressor

```
[ ] parameters = {
                  "min_samples_split": range(6, 11)}
    GCV=GridSearchCV(GradientBoostingRegressor(),parameters,cv=5)
    GCV.fit(x_train,y_train)
```

```
    GridSearchCV(cv=5, estimator=GradientBoostingRegressor(),
                 param_grid={'min_samples_split': range(6, 11)})
```

```
[ ] GCV.best_params_
```

```
    {'min_samples_split': 6}
```

```
[ ]
    mod3=GradientBoostingRegressor(min_samples_split=6)
    mod3.fit(x_train,y_train)
    pred=mod3.predict(x_test)
    print(r2_score(y_test,pred))
```

```
    0.6248441762191346
```

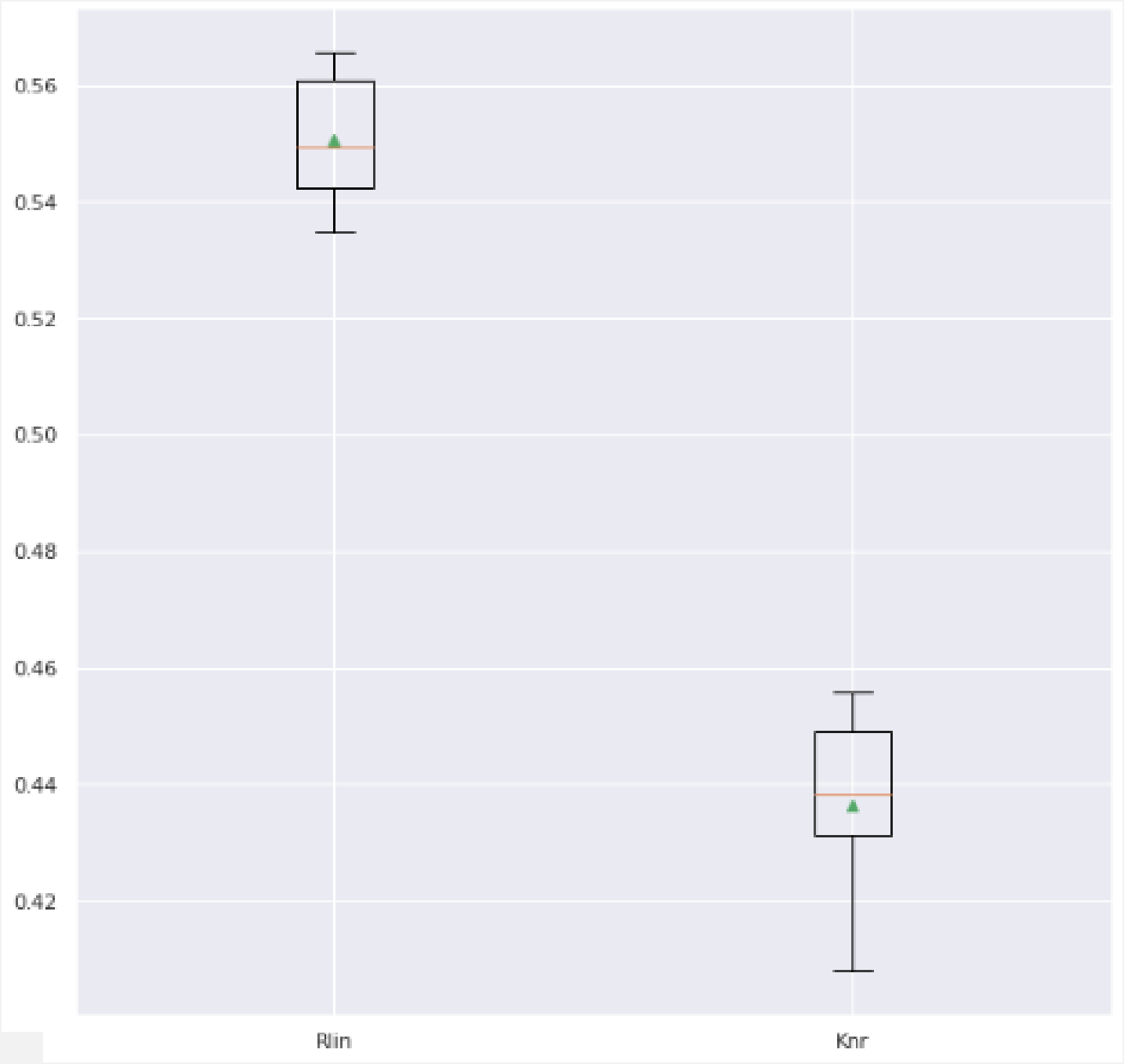
Cross Validation

Big Mart Sales Prediction

```
[147] #Models
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.linear_model import LogisticRegression
      model_rlin=LinearRegression()
      model_knn =  KNeighborsRegressor()
```

```
[150] #cross validation
      from sklearn.model_selection import RepeatedStratifiedKFold
      cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
      # evaluate the model
      from numpy import mean
      from numpy import std
      from sklearn.model_selection import cross_val_score
      scores_rlin = cross_val_score(model_rlin, x_train, y_train, scoring='r2', cv=5)
      print('>RLog %.3f (%.3f)' % (mean(scores_rlin), std(scores_rlin)))
      scores_knn = cross_val_score(model_knn, x_train, y_train, scoring='r2', cv=5)
      print('>Knn %.3f (%.3f)' % (mean(scores_knn), std(scores_knn)))
      results=[scores_rlog , scores_knn]
```

```
>RLog 0.551 (0.011)
>Knn 0.436 (0.016)
```


0.56
0.54
0.52
0.50
0.48
0.46
0.44
0.42
Rlin
Knr

```
# step-1: cross-validation
folds = KFold(n_splits = 5, shuffle = True, random_state = 100)

# step-2:  range of hyperparameters to tune
hyper_params = [{'n_features_to_select': list(range(1, 34))}]


# step-3:  grid search
# 3.1 specify model
lm = LinearRegression()
lm.fit(x_train, y_train)
rfe = RFE(lm)
# 3.2 call GridSearchCV()
model_cv = GridSearchCV(estimator = rfe,
                        param_grid = hyper_params,
                        scoring= 'r2',
                        cv = folds,
                        verbose = 1,
                        return_train_score=True)

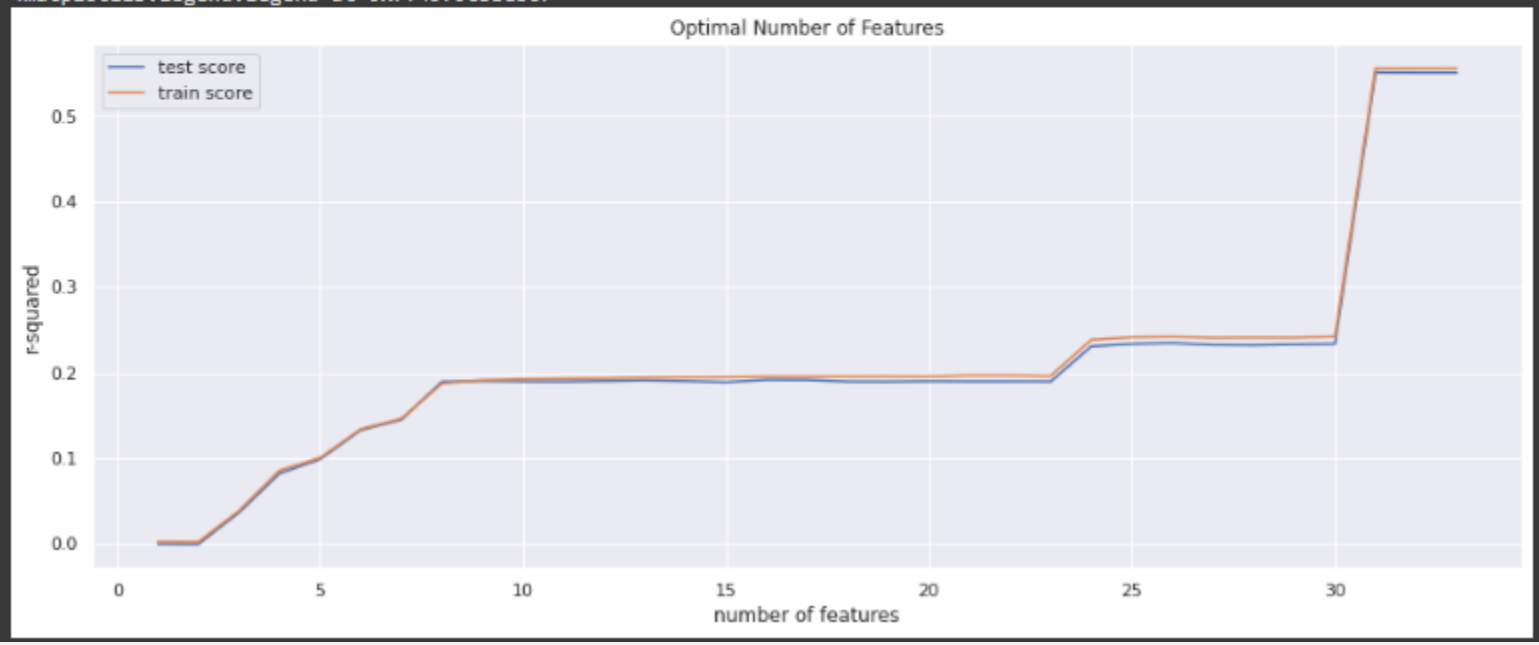
# fit the model
model_cv.fit(x_train, y_train)
```

```
Fitting 5 folds for each of 33 candidates, totalling 165 fits
GridSearchCV(cv=KFold(n_splits=5, random_state=100, shuffle=True
             estimator=RFE(estimator=LinearRegression()),
             param_grid=[{'n_features_to_select': [1, 2, 3, 4, 5
                                                  10, 11, 12, 1
                                                  17, 18, 19, 2
                                                  24, 25, 26, 2
             return_train_score=True, scoring='r2', verbose=1)
```

```
[107] # plotting cv results
      plt.figure(figsize=(16,6))

      plt.plot(cv_results["param_n_features_to_select"], cv_results["mean_test_score"])
      plt.plot(cv_results["param_n_features_to_select"], cv_results["mean_train_score"])
      plt.xlabel('number of features')
      plt.ylabel('r-squared')
      plt.title("Optimal Number of Features")
      plt.legend(['test score', 'train score'], loc='upper left')
```

```
<matplotlib.legend.Legend at 0x7f4979c3be50>
```



linear regression

```
[126] from sklearn.model_selection import cross_val_score
      lr=LinearRegression()
      score=cross_val_score(lr,x_train,y_train,cv=10,scoring='neg_mean_squared_error')
      lr_score_cross=np.sqrt(-score)
      np.mean(lr_score_cross),np.std(lr_score_cross)
```

```
(1140.5746449348496, 25.880191453178053)
```

Ridge Regression

```
[127] r=Ridge(alpha=0.05,solver='cholesky')
      score=cross_val_score(r,x_train,y_train,cv=10,scoring='neg_mean_squared_error')
      r_score_cross=np.sqrt(-score)
      np.mean(r_score_cross),np.std(r_score_cross)
```

```
(1140.3434028113452, 25.759171883489092)
```

LASSO

```
[128] l=Lasso(alpha=0.01)
      score=cross_val_score(l,x_train,y_train,cv=10,scoring='neg_mean_squared_error')
      l_score_cross=np.sqrt(-score)
      np.mean(l_score_cross),np.std(l_score_cross)
```

```
(1140.3404351398294, 25.759558237894435)
```

```
[142] go['RMSE']=go.applymap(lambda x: x.mean())
      go.RMSE.sort_values()
```

```
Gradient Boost              1057.92
Gradient Boost CV           1092.72
Linear Regression           1098.00
Lasso Regression            1098.00
Ridge Regression            1098.00
Elastic Net Regression      1098.76
SGD Regression              1107.43
Random Forest               1114.60
Ada Boost                   1124.53
Lasso Regression CV         1140.34
Ridge Regression CV         1140.34
Elastic Net Regression CV   1140.40
Linear Regression CV        1140.57
Random Forest CV            1144.13
SGD Regression CV           1146.38
Bagging                     1148.88
SVM                         1166.81
Bagging CV                  1190.33
SVM CV                      1196.62
Ada Boost CV                1287.30
Decision Tree               1509.25
Decision Tree Regression    1560.52
Name: RMSE, dtype: float64
```

```
[144] from sklearn.model_selection import GridSearchCV
      gb=GradientBoostingRegressor(max_depth=7,n_estimators=200,learning_rate=0.01)
      param=[{'min_samples_split':[5,9,13],'max_leaf_nodes':[3,5,7,9],'max_features':[8,10,15,18]}]
      gs=GridSearchCV(gb,param,cv=5,scoring='neg_mean_squared_error')
      gs.fit(x_train,y_train)
```

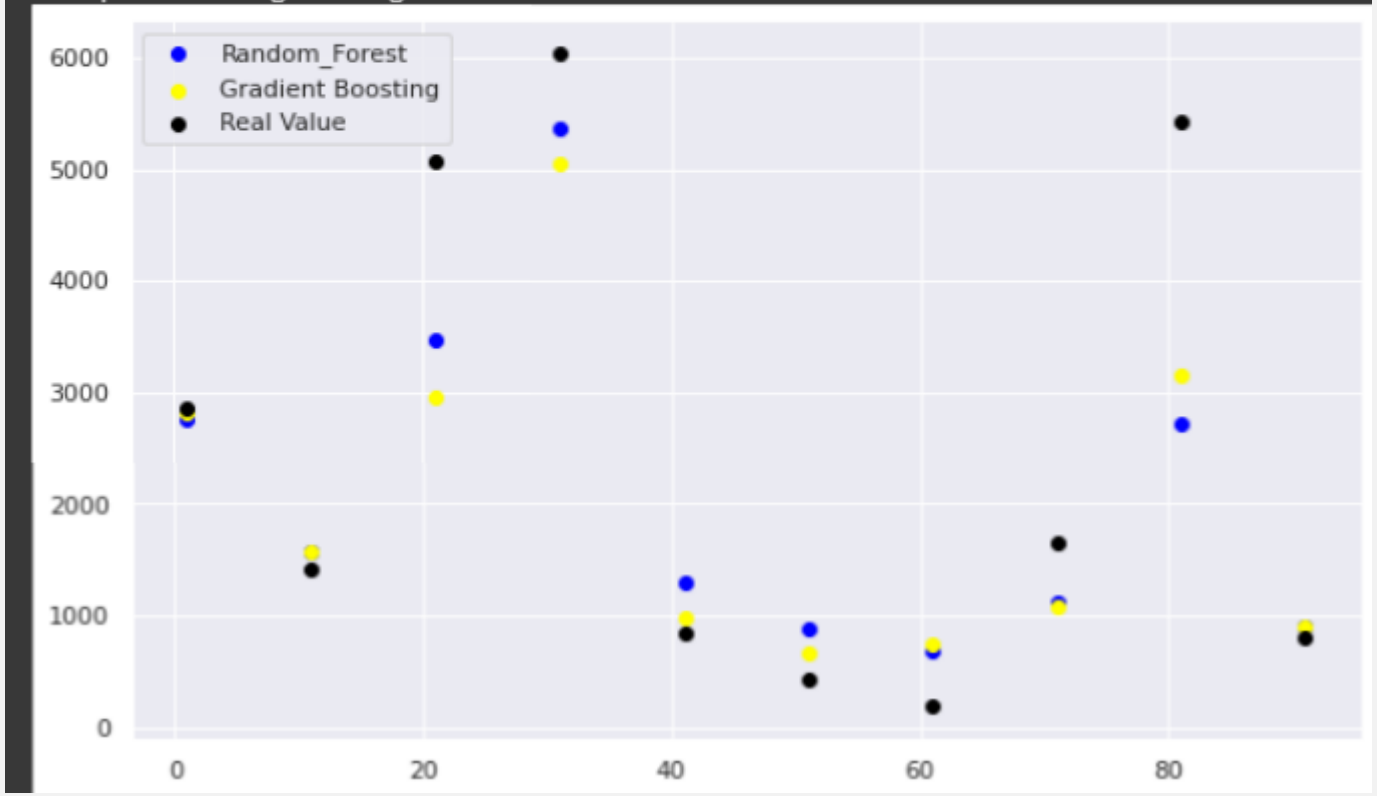
```
GridSearchCV(cv=5,
             estimator=GradientBoostingRegressor(learning_rate=0.01,
                                                 max_depth=7,
                                                 n_estimators=200),
             param_grid=[{'max_features': [8, 10, 15, 18],
                          'max_leaf_nodes': [3, 5, 7, 9],
                          'min_samples_split': [5, 9, 13]}],
             scoring='neg_mean_squared_error')
```

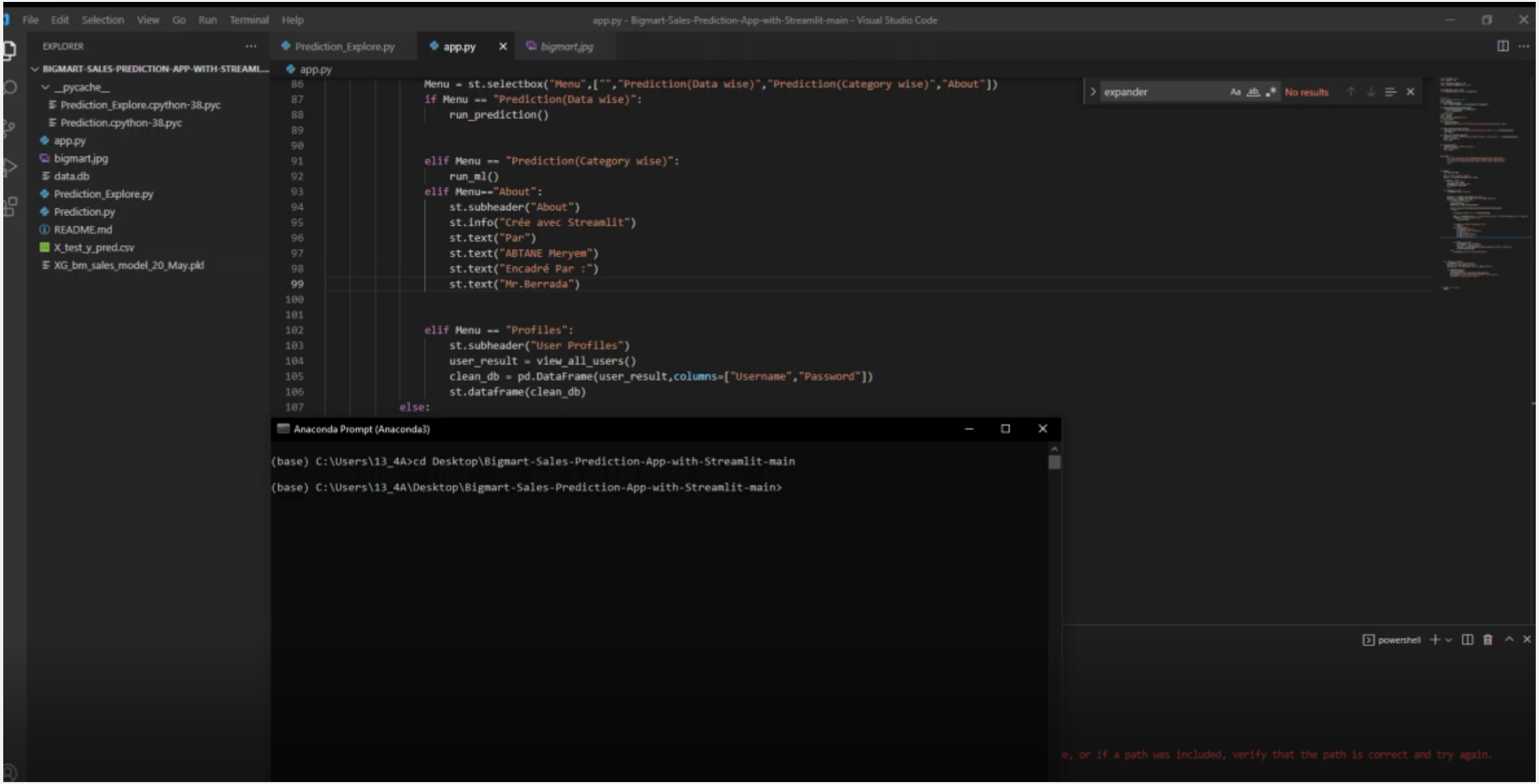
```
[145] gs.best_estimator_
```

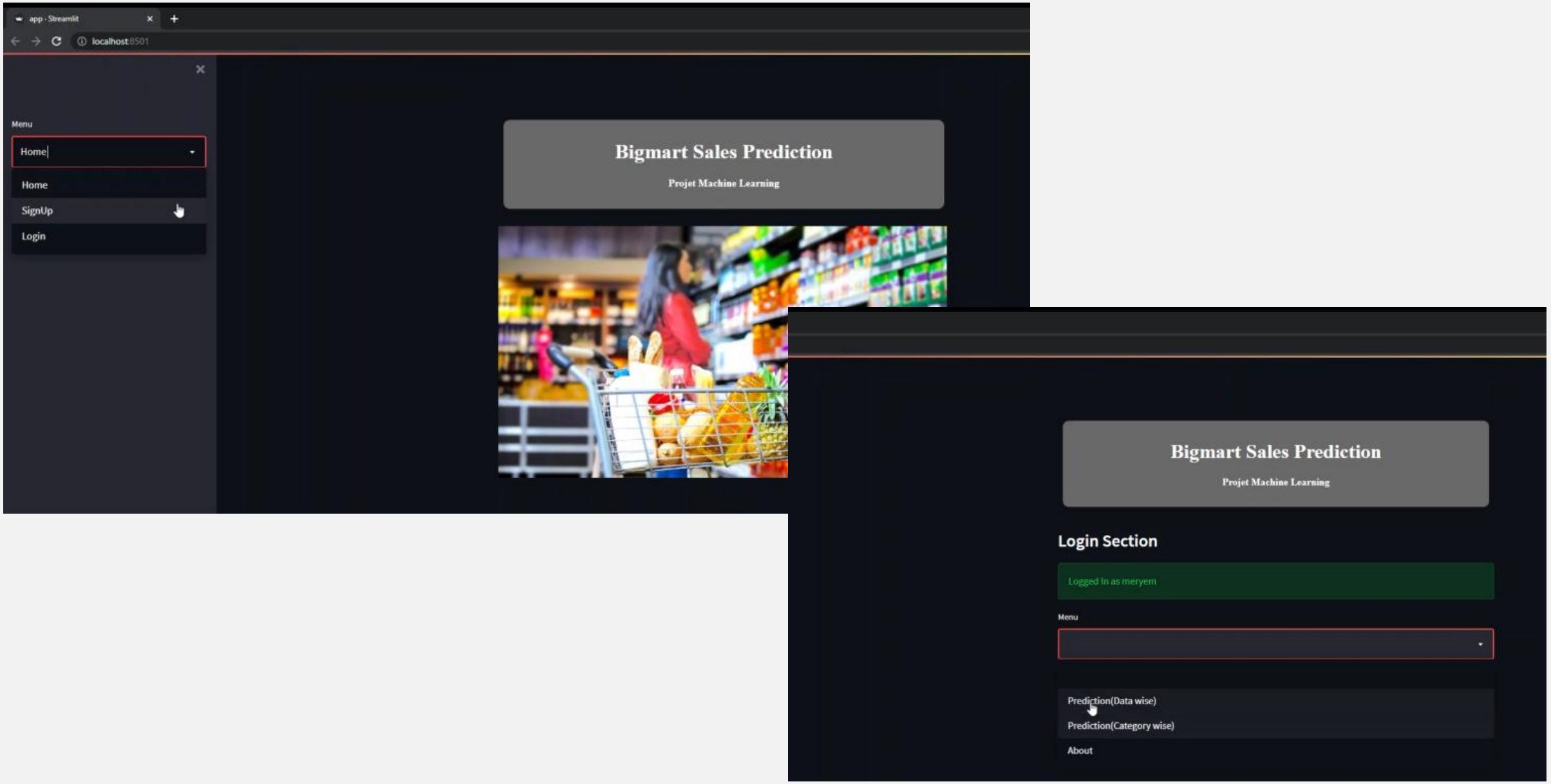
```
GradientBoostingRegressor(learning_rate=0.01, max_depth=7, max_features=18,
                          max_leaf_nodes=9, min_samples_split=9,
                          n_estimators=200)
```

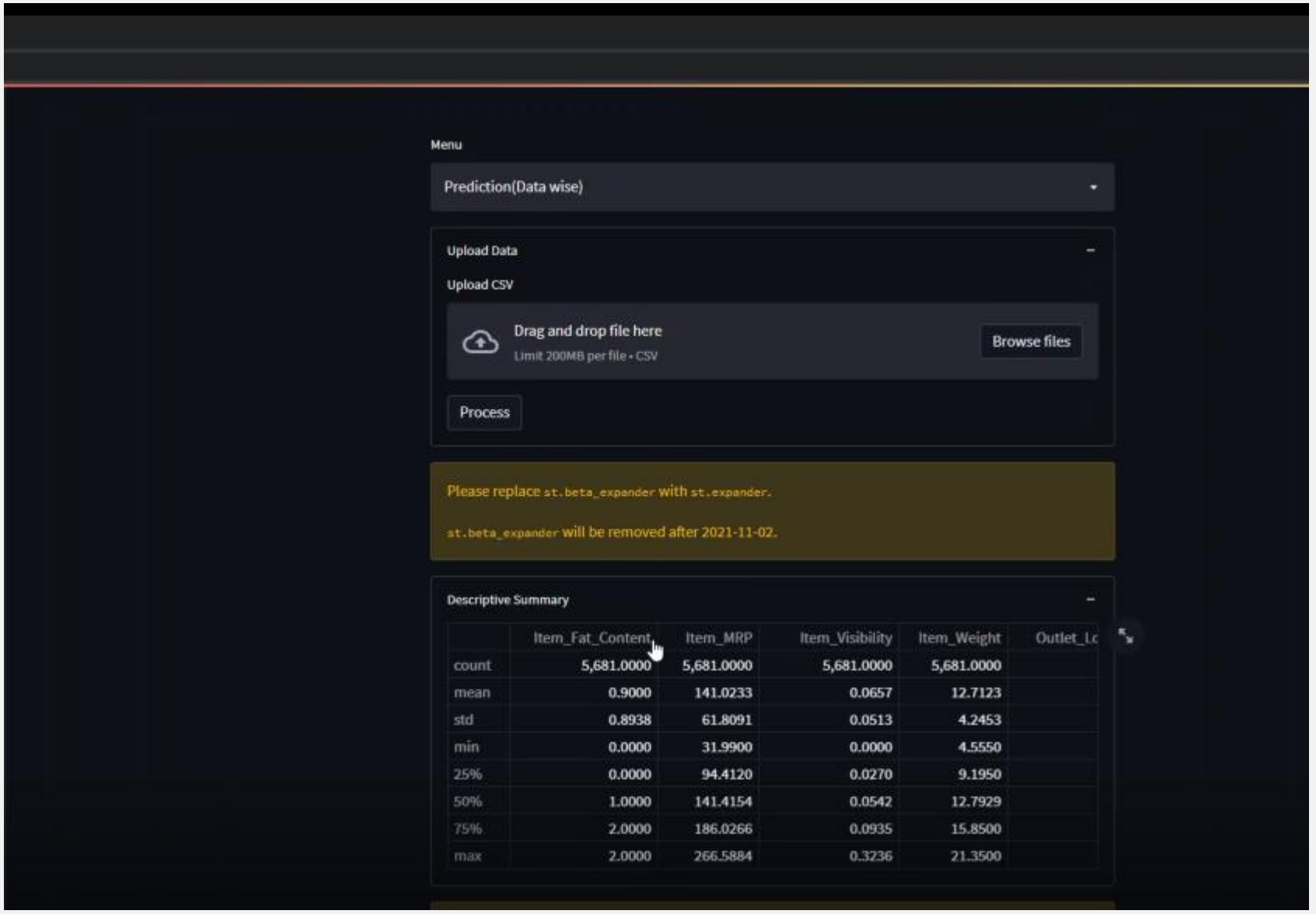
```
fig=plt.figure(figsize=(10,6))
plt.scatter(np.arange(1,100,10),predict_r[0:100:10],color='blue')
plt.scatter(np.arange(1,100,10),p[0:100:10],color='yellow')
plt.scatter(np.arange(1,100,10),y_test[0:100:10],color='black')
plt.legend(['Random_Forest','Gradient Boosting','Real Value'])
```

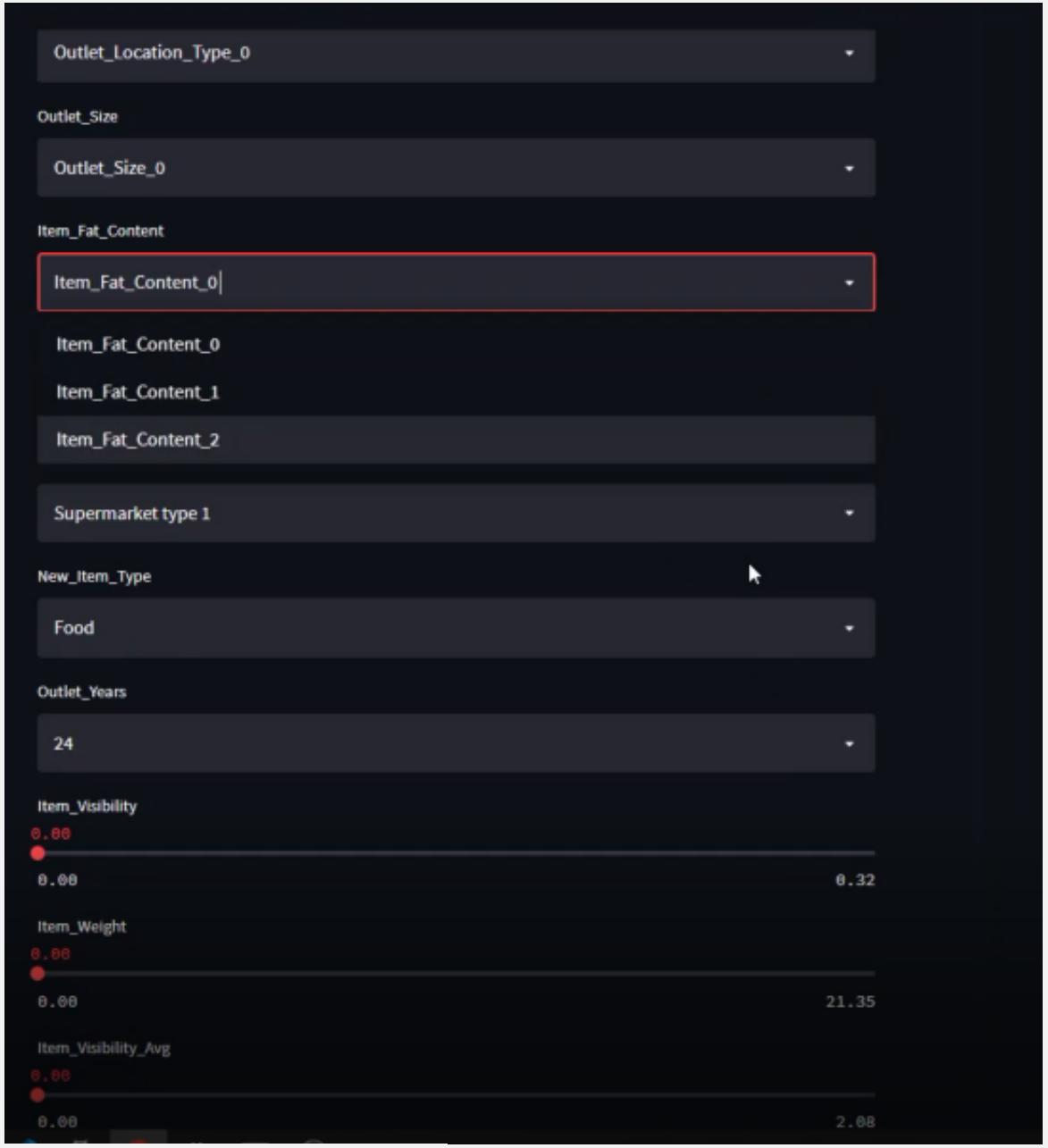
```
<matplotlib.legend.Legend at 0x7f49793feed0>
```

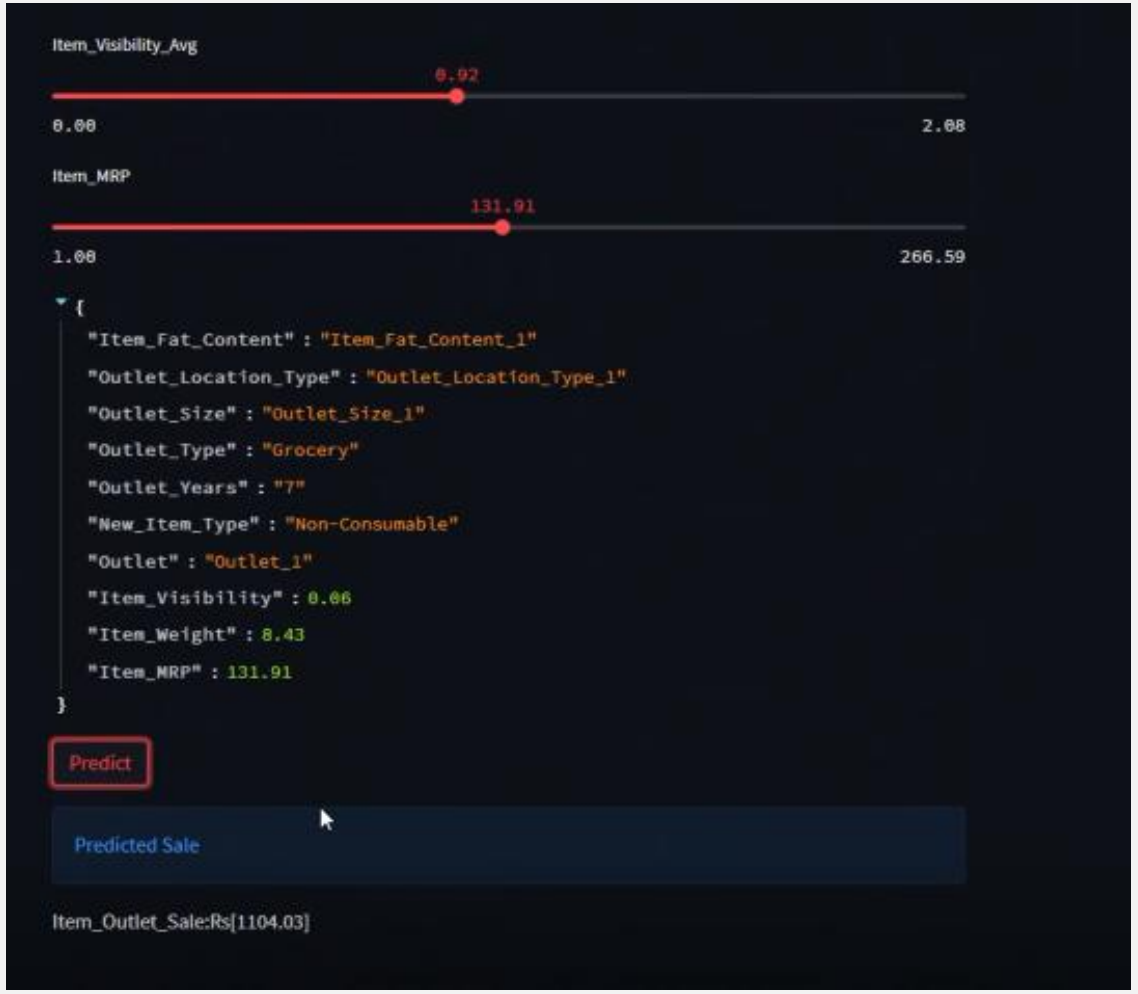



File Edit Selection View Go Run Terminal Help
app.py - Bigmart-Sales-Prediction-App-with-Streamlit-main - Visual Studio Code
EXPLORER
BIGMART-SALES-PREDICTION-APP-WITH-STREAML...
__pycache__
Prediction_Explore.cpython-38.pyc
Prediction.cpython-38.pyc
app.py
bigmart.jpg
data.db
Prediction_Explore.py
Prediction.py
README.md
X_test_y_pred.csv
XG_bm_sales_model_20_May.pkl
Prediction_Explore.py
app.py
bigmart.jpg
app.py
Menu = st.selectbox("Menu",["","Prediction(Data wise)","Prediction(Category wise)","About"])
if Menu == "Prediction(Data wise)":
run_prediction()
elif Menu == "Prediction(Category wise)":
run_ml()
elif Menu=="About":
st.subheader("About")
st.info("Crée avec Streamlit")
st.text("Par")
st.text("ABTANE Meryem")
st.text("Encadré Par :")
st.text("Mr.Berrada")
elif Menu == "Profiles":
st.subheader("User Profiles")
user_result = view_all_users()
clean_db = pd.DataFrame(user_result,columns=["Username","Password"])
st.dataframe(clean_db)
else:
expander
No results
Anaconda Prompt (Anaconda3)
(base) C:\Users\13_4A>cd Desktop\Bigmart-Sales-Prediction-App-with-Streamlit-main
(base) C:\Users\13_4A\Desktop\Bigmart-Sales-Prediction-App-with-Streamlit-main>
powershell
e, or if a path was included, verify that the path is correct and try again.


app - Streamlit
localhost:8501
Menu
Home
Home
SignUp
Login
Bigmart Sales Prediction
Projet Machine Learning
Bigmart Sales Prediction
Projet Machine Learning
Login Section
Logged In as meryem
Menu
Prediction(Data wise)
Prediction(Category wise)
About


Menu
Prediction(Data wise)
Upload Data
Upload CSV
Drag and drop file here
Limit 200MB per file • CSV
Browse files
Process
Please replace st.beta_expander with st.expander.
st.beta_expander will be removed after 2021-11-02.
Descriptive Summary
Item_Fat_Content Item_MRP Item_Visibility Item_Weight Outlet_Lc
count 5,681.0000 5,681.0000 5,681.0000 5,681.0000
mean 0.9000 141.0233 0.0657 12.7123
std 0.8938 61.8091 0.0513 4.2453
min 0.0000 31.9900 0.0000 4.5550
25% 0.0000 94.4120 0.0270 9.1950
50% 1.0000 141.4154 0.0542 12.7929
75% 2.0000 186.0266 0.0935 15.8500
max 2.0000 266.5884 0.3236 21.3500


Outlet_Location_Type_0
Outlet_Size
Outlet_Size_0
Item_Fat_Content
Item_Fat_Content_0
Item_Fat_Content_0
Item_Fat_Content_1
Item_Fat_Content_2
Supermarket type 1
New_Item_Type
Food
Outlet_Years
24
Item_Visibility
0.00
0.00
0.32
Item_Weight
0.00
0.00
21.35
Item_Visibility_Avg
0.00
0.00
2.08


Item_Visibility_Avg
0.02
0.00
2.08
Item_MRP
131.91
1.00
266.59
{
"Item_Fat_Content" : "Item_Fat_Content_1"
"Outlet_Location_Type" : "Outlet_Location_Type_1"
"Outlet_Size" : "Outlet_Size_1"
"Outlet_Type" : "Grocery"
"Outlet_Years" : "7"
"New_Item_Type" : "Non-Consumable"
"Outlet" : "Outlet_1"
"Item_Visibility" : 0.06
"Item_Weight" : 8.43
"Item_MRP" : 131.91
}
Predict
Predicted Sale
Item_Outlet_Sale:Rs[1104.03]

Merci pour votre attention